Code-To-Documentation AI System

Project Stage – II Report (CS851PC)

Submitted

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

VELISOJU ASHRITH

[Reg. No. 22261A0556]

Under the guidance of

MS. D. S. BHAVANI

(ASSISTANT PROFESSOR)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

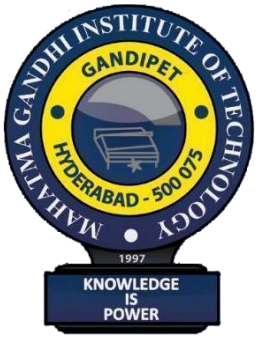
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY [A]

GANDIPET, HYDERABAD-500 075, INDIA

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad) Gandipet, Hyderabad-500 075, Telangana (India)



CERTIFICATE

This is to certify that the Project Stage – II Report (CS851PC) entitled "**Code-To-Documentation AI System",** is being submitted by **Ashrith Velisoju, Roll No: 22261A0556,** in partial fulfilment of the requirements for completion of the **Bachelor of Technology** in **Computer Science and Engineering** at **Mahatma Gandhi Institute of Technology (A),** presents a record of bona fide work carried out by them under our guidance and supervision.

The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide
Ms. D.S. Bhavani
Asst. Professor, Dept. of CSE

Head of the Department
Dr. C.R.K. Reddy
Professor, Dept. of CSE

External Examiner

DECLARATION

We hereby declare that the work described in this report, entitled **"Code-To-Documentation AI System"** which is being submitted by us in partial fulfillment for the award of **Bachelor of Technology (B.Tech.)** in Computer Science and Engineering to the Department, **Mahatma Gandhi Institute of Technology(A)**, Hyderabad-500075 is the result of investigations carried out by us under the guidance of **Ms. D.S. Bhavani,** Assistant Professor, Department of Computer Science and Engineering.

The work is original and has not been submitted for any Degree of this or any other university.

Place: Hyderabad

Date:

VELISOJU ASHRITH
(22261A0556)

ACKNOWLEDGEMENT

The satisfaction that comes with successfully completing any task is not truly complete without recognizing the people who made it possible. Success is the result of hard work and perseverance. The support and dedication of others provide valuable guidance. Therefore, I want to acknowledge everyone whose encouragement and direction illuminated my path and helped crown my efforts with success.

We want to express our sincere thanks to **Prof. G. Chandra Mohan Reddy, Principal of MGIT**, for providing the college with working facilities.

We wish to express our sincere thanks and gratitude to **Dr. C. R. K. Reddy, Professor and HOD**, Department of CSE, MGIT, for all the timely support and valuable suggestions during the project period.

We are incredibly thankful to **Dr. K. Madhu Babu, Assistant Professor, Department of CSE, MGIT, and M. Mamatha, Assistant Professor, Department of CSE, MGIT, Project Stage-II Coordinators,** for their encouragement and support throughout the project.

We want to express our sincere appreciation to our mentor, **Ms. D. S. Bhavani, Assistant Professor, Department of CSE, MGIT**, for their guidance and unwavering support, which greatly contributed to the successful completion of our work.

Finally, we would like to thank all the faculty and staff of the CSE Department who helped us, directly or indirectly, in completing the Code-To-Documentation AI System.

VELISOJU ASHRITH
(22261A0556)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

Software documentation remains one of the most persistent challenges in modern development environments, with studies indicating that inadequate documentation accounts for 60% of software maintenance costs [1] and significantly extends developer onboarding time. Existing documentation practices suffer from manual overhead, inconsistency across projects, and rapid obsolescence as codebases evolve, creating technical debt that hampers long-term software maintainability. Current automated documentation tools provide only bare API references, failing to capture architectural context, usage patterns, or business logic, and thus fail to meet the comprehensive documentation needs of complex software systems. [2], [3]

Code-To-Documentation AI System develops an intelligent documentation generation system that leverages large language models trained on extensive code-documentation pairs [13], [14] to automatically produce comprehensive, contextually aware documentation from source code analysis. The system incorporates multilingual code analyzers, architectural visualization generators, and change-detection mechanisms that automatically flag outdated documentation sections, ensuring documentation remains synchronized with evolving codebases. Through interactive documentation interfaces with verification capabilities and intelligent example generation, the platform addresses the full spectrum of documentation needs from API references to architectural overviews. The implementation promises to dramatically reduce documentation maintenance overhead while improving software quality, accelerating developer productivity, and facilitating knowledge transfer in distributed development teams.

1. INTRODUCTION

Software documentation plays a critical role in the development lifecycle, influencing maintainability, developer onboarding, collaboration efficiency, and long-term project sustainability. Despite its importance, documentation remains one of the most neglected aspects of software engineering. [1], [2] Many development teams struggle with outdated, incomplete, or inconsistent documentation due to rapid codebase changes, time constraints, and the manual effort required to maintain detailed technical records. As a result, insufficient documentation significantly increases maintenance costs and reduces productivity across software organizations. [1]

The Code-to-Software Documentation AI System explores the use of artificial intelligence and large language models to address persistent challenges in software documentation. By analyzing source code across various programming languages, the system aims to automatically generate comprehensive, context-aware documentation that captures not only API details but also architectural insights, usage patterns, and underlying business logic. The project centers on advanced natural language processing (NLP) models trained on extensive code documentation datasets, [13] enabling the system to understand code semantics and produce high-quality documentation that evolves with the codebase. [7], [8]

The system architecture includes modules for automated code parsing, dependency analysis, documentation generation, architectural visualization, and change detection. [18] Source code is collected from repositories, processed with multilingual analyzers, and mapped to structured representations that serve as input to the AI model. The generated documentation is enriched with visual diagrams, flow representations, and example usage snippets to provide developers with a deeper understanding of system behavior and component interactions.

While the tool is not meant to replace human oversight in critical design decisions, it demonstrates how artificial intelligence can serve as a powerful complement to modern software development workflows. By reducing manual effort and improving documentation quality, the Code-to-Software Documentation AI System represents a significant advancement in applying AI-driven automation to real-world engineering challenges, ultimately enhancing developer productivity and promoting long-term software reliability.

1.1 Problem Definition

Software systems are becoming increasingly complex, involving large codebases, multiple programming languages, distributed teams, and frequent updates. In such environments, maintaining high-quality software documentation remains a persistent challenge. [1], [2] Despite advancements in development tools, producing accurate, comprehensive, and up-to-date documentation remains manual, mainly leading to inconsistency, technical debt, and significant onboarding delays. Traditional documentation practices rely on developer-written comments and static tools, which often fail to capture evolving architectural context, business logic, or real-world usage patterns. [2], [3]

The main problem Code-To-Documentation AI System addresses is the lack of an intelligent, automated, and context-aware documentation system that can understand code semantically and generate complete, scalable documentation. Existing tools generate only bare API-level references but cannot understand deeper functional relationships or detect when documentation becomes outdated. [4], [5] As software evolves rapidly, documentation quickly becomes obsolete, rendering it unreliable for developers, testers, and stakeholders.

Therefore, the objective is to design an AI-powered system that can:

1. Analyze source code across programming languages to understand structure, logic, and architectural relationships. [8], [10]

2. Automatically generate comprehensive, context-rich documentation, including descriptions, workflows, and architectural insights. [14], [18]

3. Detect changes in the codebase and intelligently flag outdated documentation. [12]

4. Create visual representations of system architecture to enable faster comprehension and knowledge transfer. [18]

By leveraging Large Language Models trained on diverse code-documentation pairs [19], the system aims to determine whether intelligent automation can bridge the documentation gap in modern software engineering. The goal is not merely to replace human-written documentation, but to enhance maintainability, reduce developer effort, and improve documentation reliability by providing an adaptive, data-driven, and continuously updated solution that supports long-term software sustainability and team productivity. Unpredictable natural disasters can cause severe damage to infrastructure, the environment, and human life.

1.2 Existing Applications

Current automated documentation tools in the software development ecosystem provide only bare API references and function-level documentation, failing to capture the full software context. These existing systems rely on simple code parsing and template-based generation that extracts method signatures, parameter lists, and basic descriptions directly from code comments. [1], [4] The current approach relies heavily on developers' manual documentation efforts, resulting in inconsistent documentation standards across projects and teams. [2] Traditional documentation tools lack integration with version control systems and cannot automatically detect when code changes render existing documentation obsolete. [12] The existing systems also fail to understand architectural relationships, business-logic context, or usage patterns, limiting their utility to surface-level reference material rather than to meaningful guidance for developers. [3], [5]

1.3 Proposed Application

The proposed intelligent documentation generation system introduces a comprehensive AI-driven approach that leverages large language models trained on extensive code-documentation pairs [13], [14] to automatically produce contextually aware documentation from source code analysis. The system incorporates Multi-Language Code Analyzers that can parse and understand code structures across different programming languages, extracting not only syntax but also semantic meaning and architectural patterns. [10] Context-Aware Documentation Generators utilize advanced NLP models to understand business logic, usage patterns, and architectural relationships, producing documentation that explains not just what the code does, but why and how it fits into the larger system context. [13], [19] Automated Change Detection Mechanisms continuously monitor codebase evolution and automatically flag outdated documentation sections, ensuring documentation remains synchronized with the evolving code. [12] Architectural Visualization Generators create dynamic visual representations of system architecture, data flows, and component relationships, making complex systems more comprehensible. [18] Interactive Documentation Interfaces with verification capabilities allow developers to validate, update, and enhance generated documentation through intelligent feedback loops. [17] The system also features Intelligent Example Generation, which automatically generates relevant code examples and usage scenarios, dramatically reducing documentation maintenance overhead while improving software quality, accelerating

developer productivity, and facilitating knowledge transfer across distributed development teams. [14], [18]

1.4 Requirements Specification

Requirement Specifications describe the art and craft of Software Requirements and Hardware Requirements used in the Code-To-Documentation AI System.

Software Requirements

Operating System: Windows 10 / Linux / macOS

Programming Language: TypeScript 5.6, JavaScript (Node.js 18+)

Frontend Frameworks: React 18, Vite 7, Tailwind CSS 3.4, Shadcn/UI, Framer Motion, Mermaid.js

Backend Frameworks: Express 5, Mongoose 9 (ODM), Passport.js, Zod, Simple-Git

AI Provider SDKs: Salesforce, Groq SDK, OpenRouter (via OpenAI SDK), Bytez.js, Ollama REST API

Document Generation: docx library (Word export), Puppeteer (Mermaid-to-PNG rendering)

IDE / Code Editor: VS Code / WebStorm

Database: MongoDB (Local or MongoDB Atlas)

Authentication: GitHub OAuth 2.0

Others: npm, cross-env, tsx, esbuild

Hardware Requirements

Processor: Intel i5 or higher / AMD equivalent

RAM: Minimum 8 GB (16 GB recommended for Puppeteer rendering)

Storage: Minimum 2 GB of free disk space

Internet: Required for GitHub OAuth, AI provider APIs (Groq, OpenRouter, Bytez)

Browser: Chromium-compatible browser (used internally by Puppeteer)

2. LITERATURE SURVEY

[1] Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, K Vijay-Shanker (2011), "Towards Automatically Generating Summary Comments for Java Methods"

Introduces template-based natural language generation for Java methods. Establishes a foundational methodology for automatic summarisation. Research contribution includes a systematic approach to method-level documentation and natural language generation.

[2] Sonia Haiduc, Jairo Aponte, Adrian Marcus (2012), "Supporting Program Comprehension with Source Code Summarisation"

Establishes template-based generation with program comprehension focus. Demonstrates a practical framework for documentation support. Research impact includes foundational template methods and applications in vector-space modeling.

[3] Sonia Haiduc, Jairo Aponte, Laura Moreno, and Adrian Marcus (2013), "On the Use of Automated Text Summarisation Techniques for Summarising Source Code"

Introduces information retrieval techniques for code summarisation. Establishes early automation approaches. Research contribution includes foundational IR techniques and automated text processing for code.

[4] Laura Moreno, Jairo Aponte, Giriprasad Sridhara, Andrian Marcus, Lori Pollock, K Vijay-Shanker (2014), "Automatic Generation of Natural Language Summaries for Java Classes"

Establishes a heuristic approach for Java class documentation. Demonstrates practical implementation of automated summarisation. Research impact includes foundational methodology and rule-based processing techniques.

[5] Nahla J. Abid, Natalia Dragan, Michael L. Collard, Jonathan I. Maletic (2015), "Using Stereotypes in the Automatic Generation of Natural Language Summaries for C++ Methods"

Introduces a stereotype-based approach for C++ method summarisation. Demonstrates a structured approach to code documentation. Research contributions include systematic methodologies and framework-based processing.

[6] Miltiadis Allamanis, Hao Peng, Charles Sutton (2017), "A Convolutional Attention Network for Extreme Summarisation of Source Code"

Introduces convolutional attention for extreme code summarisation. Demonstrates innovative architecture for code understanding. Research contribution includes novel attention mechanisms and convolutional approaches to code processing.

[7] Xing Hu, Ge Li, Xin Xia, David Lo, Zhi Jin (2018), "Deep Code Comment Generation" Adapts neural machine translation for code documentation.

Introduces a structure-based traversal method for AST processing. Future research should focus on improving structural representations and handling larger code contexts.

[8] Uri Alon, Shaked Brody, Omer Levy, Eran Yahav (2019), "code2seq: Generating Sequences from Structured Representations of Code."

Introduces an AST-based path representation for code understanding. Achieves significant improvements through structural encoding. Future work should investigate more sophisticated path-selection and multi-path-integration strategies.

[9] Patrick Fernandes, Miltiadis Allamanis, Marc Brockschmidt (2019), "Structured Neural Summarisation"

Combines sequence and graph models to improve summarisation. Demonstrates the effectiveness of hybrid architectures. Research direction should explore more sophisticated fusion mechanisms and graph construction techniques.

[10] Alexander LeClair, Sakib Haque, Lingfei Wu, Collin McMillan (2020), "Improved Code Summarisation via a Graph Neural Network"

Introduces graph neural networks for improved code representation. Demonstrates the effectiveness of structural information utilization. Research direction should explore more sophisticated graph architectures and multi-scale representations.

[11] Triet H.M. Le, Hao Chen, Muhammad Ali Babar (2020), "Deep Learning for Source Code Modeling and Generation"

Provides a comprehensive survey of deep learning approaches for code modeling. Establishes taxonomy and identifies research directions. Future work should focus on emerging techniques and practical deployment considerations.

[12] Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, Xiaoyin Che, Zhiyuan Liu, Maosong Sun (2021), "RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code Documentation Generation"

Addresses repository-level documentation with a proactive generation approach. Demonstrates comprehensive documentation automation capabilities. Research direction should explore integration with version control systems and collaborative development environments.

[13] Yue Wang, Weishi Wang, Shafiq Joty, Steven C.H. Hoi (2021), "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation"

Establishes a unified architecture for code understanding and generation. Provides a strong baseline with identifier-aware training. Future development should focus on structural awareness and multi-modal capabilities.

[14] Junaed Younus Khan, Gias Uddin (2022), "Automatic Code Documentation Generation Using GPT-3"

Pioneering work in applying GPT-3 to code documentation. Demonstrates state-of-the-art performance with minimal training. Future research should investigate the capabilities of few-shot learning and fine-tuning to enhance performance.

[15] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, Gabriele Bavota (2022), "Evaluating Code Summarisation Techniques: A New Metric and an Empirical Characterization"

Introduces a comprehensive evaluation framework with novel metrics. Provides a systematic assessment of code summarisation techniques. Future work should focus on expanding evaluation dimensions and establishing standardized benchmarks.

[16] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Shaomeng Cao, Kechi Zhang, Zhi Jin (2023), "Interpretation-based Code Summarisation"

Introduces a human-inspired learning paradigm with model interpretation. Achieves significant performance improvements through a two-stage approach. Future work should investigate the accuracy of interpretation and its broader applicability across different model architectures.

[17] David Ding, Aviv Dotan, Oded Margalit, Ori Levy (2023), "AI-Mediated Code Comment Improvement"

Focuses on documentation quality improvement rather than generation. Demonstrates a practical approach to enhancing existing documentation. The research direction should explore automated quality assessment and its integration with development tools.

[18] Dayu Yang, Antoine Simoulin, Xin Qian, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Grey Yang (2024), "DocAgent: A Multi-Agent System for Automated Code Documentation Generation"

Addresses complex repository documentation with a multi-agent architecture. Demonstrates significant advancement in automated documentation through collaborative AI systems. Future work should focus on scalability and integration with development workflows.

[19] Swapnil Sharma Sarker, Tanzina Taher Ifty (2024), "Automated and Context-Aware Code Documentation Leveraging Advanced LLMs"

Focuses on modern Java documentation with advanced LLM techniques. Provides a comprehensive comparison of LLM approaches for code documentation. Research direction should explore cross-language applicability and real-world deployment scenarios.

[20] Hanzhen Lu, Zhongxin Liu (2024), "Improving Retrieval-Augmented Code Comment Generation by Retrieving for Generation"

Introduces a joint training paradigm for retrieval-augmented generation. Demonstrates superior performance through a synergistic training approach. Future research should investigate the applicability to other code intelligence tasks and scalability improvements.

Table 2.1: Literature Survey

Sno	Year	Title	Authors	Methodology	Merits	Demerits
1	2011	Towards Automatically Generating Summary Comments for Java Methods	Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, K Vijay-Shanker	Template-based natural language generation for Java method summary creation	Java method focus, template-based approach, natural language generation, foundational methodology	Limitations of Templates, Java Methods and Natural anguage Generation
2	2012	Supporting Program Comprehension with Source Code Summarisation	Sonia Haiduc, Jairo Aponte, and Andrian Marcus	Template-based approach with vector space modeling for program comprehension support	Template-based generation, vector space modeling, program comprehension support, and a practical framework	Template rigidity, vector space limitations, program comprehension scope constraints, and scalability issues
3	2013	On the Use of Automated Text Summarisation Techniques for Summarising Source Code	Sonia Haiduc, Jairo Aponte, Laura Moreno, Adrian Marcus	Information retrieval techniques, including LSI and VSM, for code summarisation	Information retrieval foundation, LSI and VSM techniques, automated summarisation, and early automation	Information retrieval limitations, LSI and VSM constraints, and automated summarisation accuracy issues
4	2014	Automatic Generation of Natural Language Summaries for Java Classes	Laura Moreno, Jairo Aponte, Giriprasad Sridhara, and Andrian Marcus	Heuristic-based approach for automatic generation of natural language summaries	Java class focus, heuristic approach, natural language generation, practical implementation	Heuristic limitations, Java class restrictions, rule-based constraints, and limited adaptability
5	2015	Using Stereotypes in the Automatic Generation of Natural Language Summaries for C++ Methods	Nahla J. Abid, Natalia Dragan, Michael L. Collard, Jonathan I. Maletic	Stereotype-based approach using the srcML framework for C++ method summarisation	Stereotype utilization, C++ method focus, srcML framework, structured approach	Stereotype dependency, C++ limitation, srcML framework constraints, and limited generalizability

Sno	Year	Title	Authors	Methodology	Merits	Demerits
6	2017	A Convolutional Attention Network for Extreme Summarisation of Source Code	Miltiadis Allamanis, Hao Peng, Charles Sutton	Convolutional attention network for extreme code summarisation with attention mechanisms	Convolutional attention, extreme summarisation capability, attention mechanisms, innovative architecture	Convolutional limitations, extreme summarisation focus, attention mechanism constraints, and limited scope
7	2018	Deep Code Comment Generation	Xing Hu, Ge Li, Xin Xia, David Lo, Zhi Jin	Structure-based traversal of AST with sequence-to-sequence neural machine translation	Structural information utilization, SBT traversal method, NMT adaptation, and substantial improvements	SBT method complexity, NMT adaptation limitations, vocabulary challenges, and structural dependency
8	2019	code2seq: Generating Sequences from Structured Representations of Code	Uri Alon, Shaked Brody, Omer Levy, Eran Yahav	AST path-based representation with an attention mechanism for sequence generation from structured code	Structural code representation, AST path encoding, attention-based selection, and significant improvements	Path representation limitations, AST dependency, computational complexity, and limited applicability
9	2019	Structured Neural Summarisation	Patrick Fernandes, Miltiadis Allamanis, Marc Brockschmidt	Hybrid sequence-graph models combining neural networks with graph components for summarisation	Hybrid architecture, long-distance relationship modeling, graph neural networks, and summarisation effectiveness	Hybrid model complexity, graph construction overhead, limited evaluation scope, and architecture complexity
10	2020	Improved Code Summarisation via a Graph Neural Network	Alexander LeClair, Sakib Haque, Lingfei Wu, Collin McMillan	Graph neural network combined with source code sequence encoding for improved summarisation	Graph structure utilization, improved AST representation, attention mechanisms, and performance gains	Graph complexity, computational overhead, dataset dependency, and limited structural diversity

Sno	Year	Title	Authors	Methodology	Merits	Demerits
11	2020	Deep Learning for Source Code Modeling and Generation	Triet H.M. Le, Hao Chen, Muhammad Ali Babar	Comprehensive survey of deep learning techniques for source code modeling and generation	Comprehensive survey, systematic categorization, deep learning focus, research direction guidance	Survey limitations, no novel methodology, limited practical contributions, and a review nature.
12	2021	RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code Documentation Generation	Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin	Large language model-powered framework for proactive repository-level documentation generation	Repository-level scope, proactive generation, comprehensive documentation, LLM-powered automation	Repository-specific limitations, LLM dependency, scalability concerns, and maintenance requirements
13	2021	CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation	Yue Wang, Weishi Wang, Shafiq Joty, Steven C.H. Hoi	Unified pre-trained encoder-decoder transformer with identifier-aware training objectives	Unified architecture, identifier-aware training, strong baseline performance, versatile applications	Identifier dependency, limited structural awareness, computational requirements, and training complexity
14	2022	Automatic Code Documentation Generation Using GPT-3	Junaed Younus Khan, Gias Uddin	Zero-shot and one-shot learning with GPT-3 Codex for documentation generation	State-of-the-art performance, minimal training requirements, multi-language support, GPT-3 effectiveness	Limited sample size evaluation, API dependency, cost considerations, parameter sensitivity
15	2022	Evaluating Code Summarisation Techniques: A New Metric and an Empirical Characterization	Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco	Comprehensive evaluation framework with a new SIDE metric for code summarisation assessment	New evaluation metric, comprehensive empirical study, robust assessment framework, practical insights	Limited baseline comparisons, metric validation requirements, and evaluation framework complexity

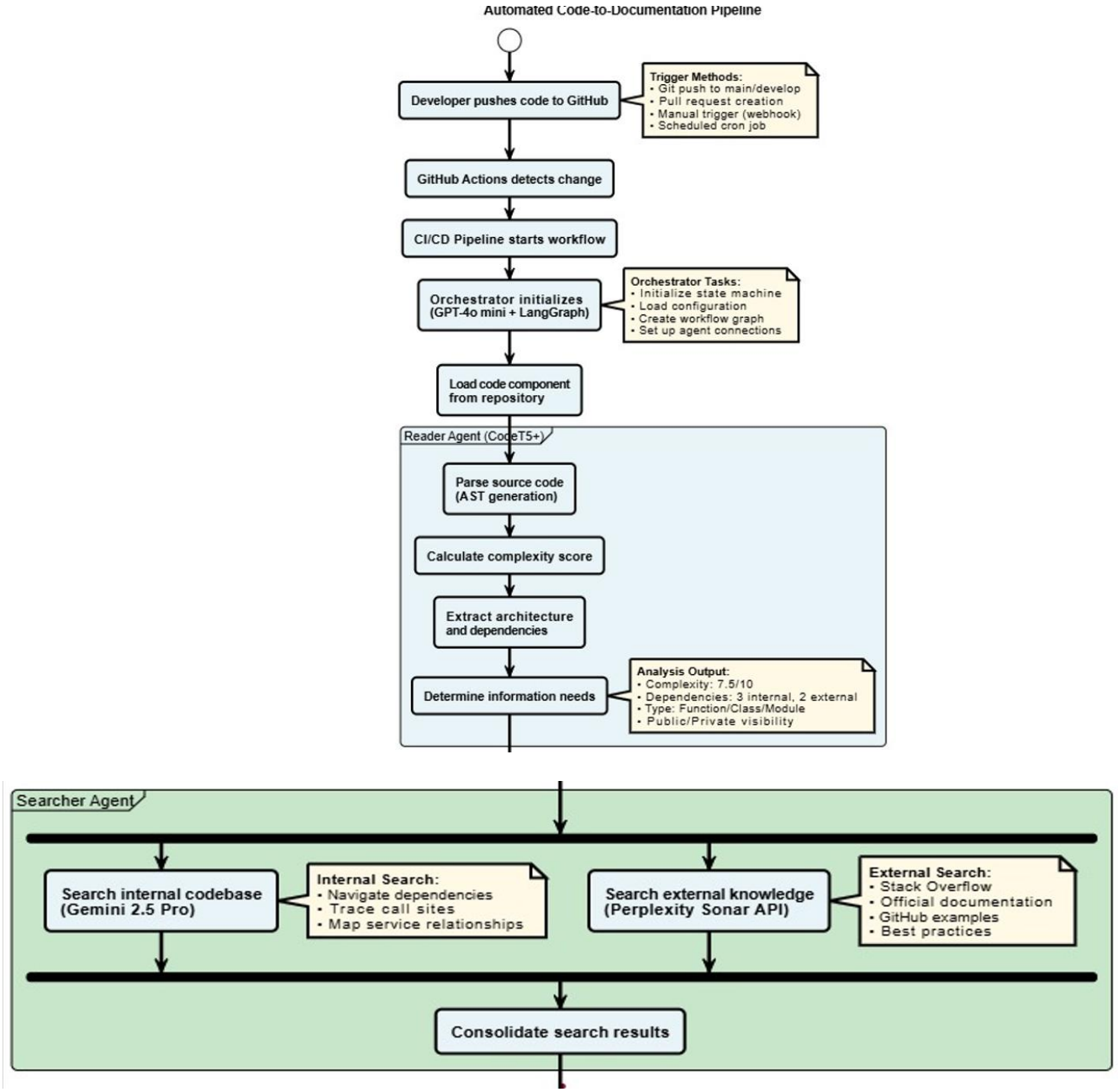
Sno	Year	Title	Authors	Methodology	Merits	Demerits
16	2023	Interpretation-based Code Summarisation	Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Shaomeng Cao, Kechi Zhang	Two-stage paradigm using model interpretation to identify focuses and reinforce learning	Model interpretation insights, significant performance improvements, and a human-inspired learning paradigm	Two-stage complexity, interpretation accuracy dependency, computational overhead, and limited generalizability
17	2023	AI-Mediated Code Comment Improvement	David Ding, Aviv Dotan, Oded Margalit, Ori Levy	AI-mediated procedure for improving comments using fine-tuned LLM and distilled models	Quality improvement focus, multi-axis enhancement, LLM integration, practical applicability	Subjective quality assessment, limited scope evaluation, dependency on human feedback
18	2024	DocAgent: A Multi-Agent System for Automated Code Documentation Generation	Dayu Yang, Antoine Simoulin, Xin Qian, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Grey Yang	Multi-agent collaborative system using topological code processing for incremental context building	Multi-agent collaboration, topological processing, a comprehensive evaluation framework, and high accuracy	Computational complexity, multi-agent coordination overhead, dependency on high-quality data

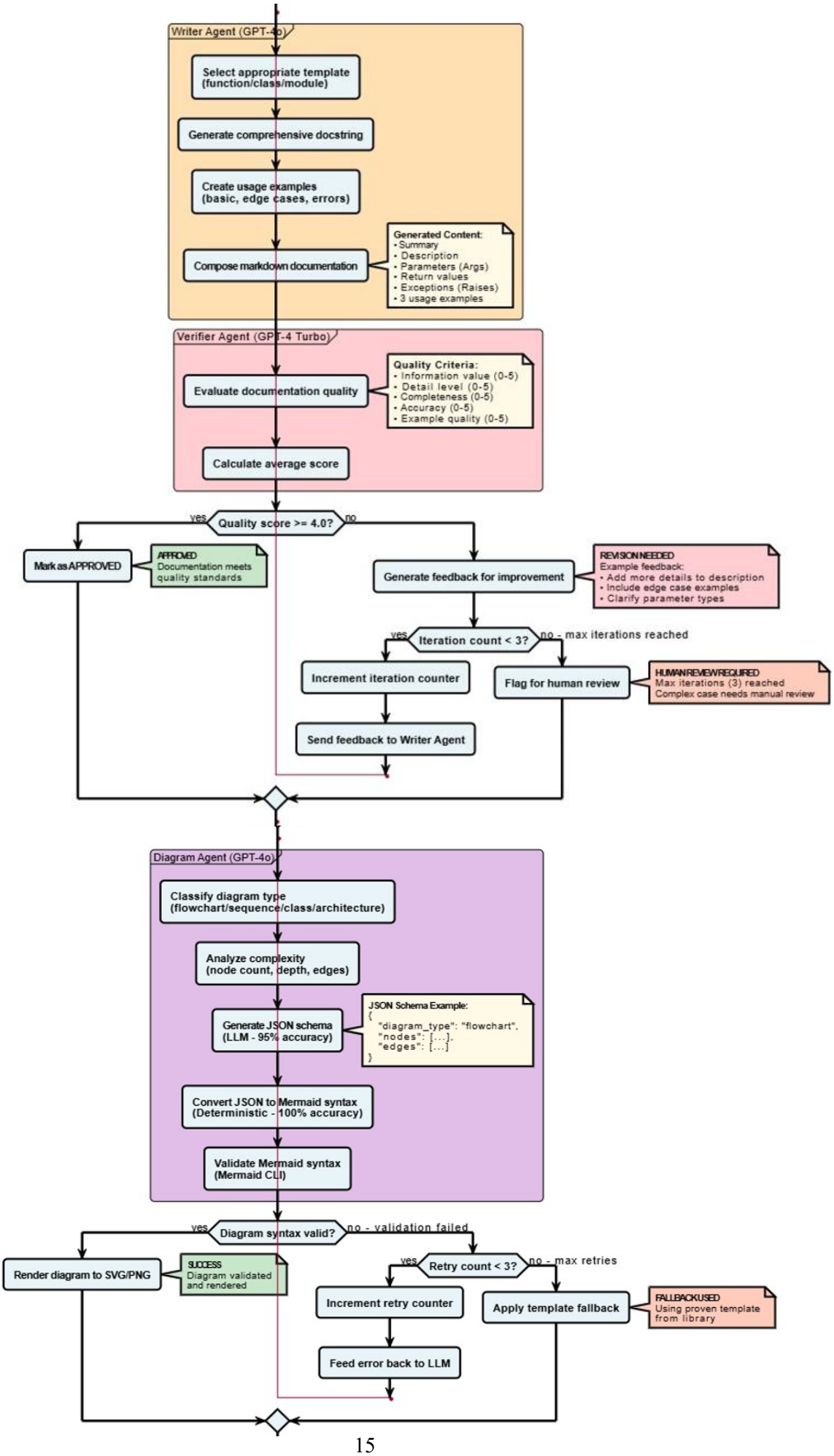
Sno	Year	Title	Authors	Methodology	Merits	Demerits
19	2024	Automated and Context-Aware Code Documentation Leveraging Advanced LLMs	Swapnil Sharma Sarker, Tanzina Taher Ifty	Contrastive pre-training with CodeT5 backbone for context-aware Javadoc generation	Context-aware generation, support for modern Java features, comparative LLM analysis, and practical applicability.	Limited to Java language, dataset dependency, and computational resource requirements
20	2024	Improving Retrieval-Augmented Code Comment Generation by Retrieving for Generation	Hanzhen Lu, Zhongxin Liu	Joint training of retriever and generator with a weighted loss function for retrieval-augmented comment generation	Joint training synergy, improved retrieval quality, robust generator performance, state-of-the-art results	Training complexity, computational overhead, retrieval base dependency, joint optimization challenges

3. DESIGN AND METHODOLOGY

The design and methodology for the Multi-Agent code-to-software documentation AI system consists of the system architecture, AI-driven workflow techniques, quality verification mechanisms, and UML diagrams that describe how the Code-to-Software Documentation AI System transforms a GitHub repository into professional documentation.

3.1 Flowchart




Writer Agent (GPT-4o)
Select appropriate template (function/class/module)
Generate comprehensive docstring
Create usage examples (basic, edge cases, errors)
Compose markdown documentation
Generated Content:
• Summary
• Description
• Parameters (Args)
• Return values
• Exceptions (Raises)
• 3 usage examples
Verifier Agent (GPT-4 Turbo)
Evaluate documentation quality
Quality Criteria:
• Information value (0-5)
• Detail level (0-5)
• Completeness (0-5)
• Accuracy (0-5)
• Example quality (0-5)
Calculate average score
Quality score >= 4.0?
yes
no
Mark as APPROVED
APPROVED
Documentation meets quality standards
Generate feedback for improvement
REVISION NEEDED
Example feedback:
• Add more details to description
• Include edge case examples
• Clarify parameter types
Iteration count < 3?
yes
no - max iterations reached
Increment iteration counter
Flag for human review
HUMAN REVIEW REQUIRED
Max iterations (3) reached
Complex case needs manual review
Send feedback to Writer Agent
Diagram Agent (GPT-4o)
Classify diagram type (flowchart/sequence/class/architecture)
Analyze complexity (node count, depth, edges)
Generate JSON schema (LLM - 95% accuracy)
JSON Schema Example:
{
"diagram_type": "flowchart",
"nodes": [...],
"edges": [...]
}
Convert JSON to Mermaid syntax (Deterministic - 100% accuracy)
Validate Mermaid syntax (Mermaid CLI)
Diagram syntax valid?
yes
no - validation failed
Render diagram to SVG/PNG
SUCCESS
Diagram validated and rendered
Retry count < 3?
yes
no - max retries
Increment retry counter
Apply template fallback
FALLBACK USED
Using proven template from library
Feed error back to LLM

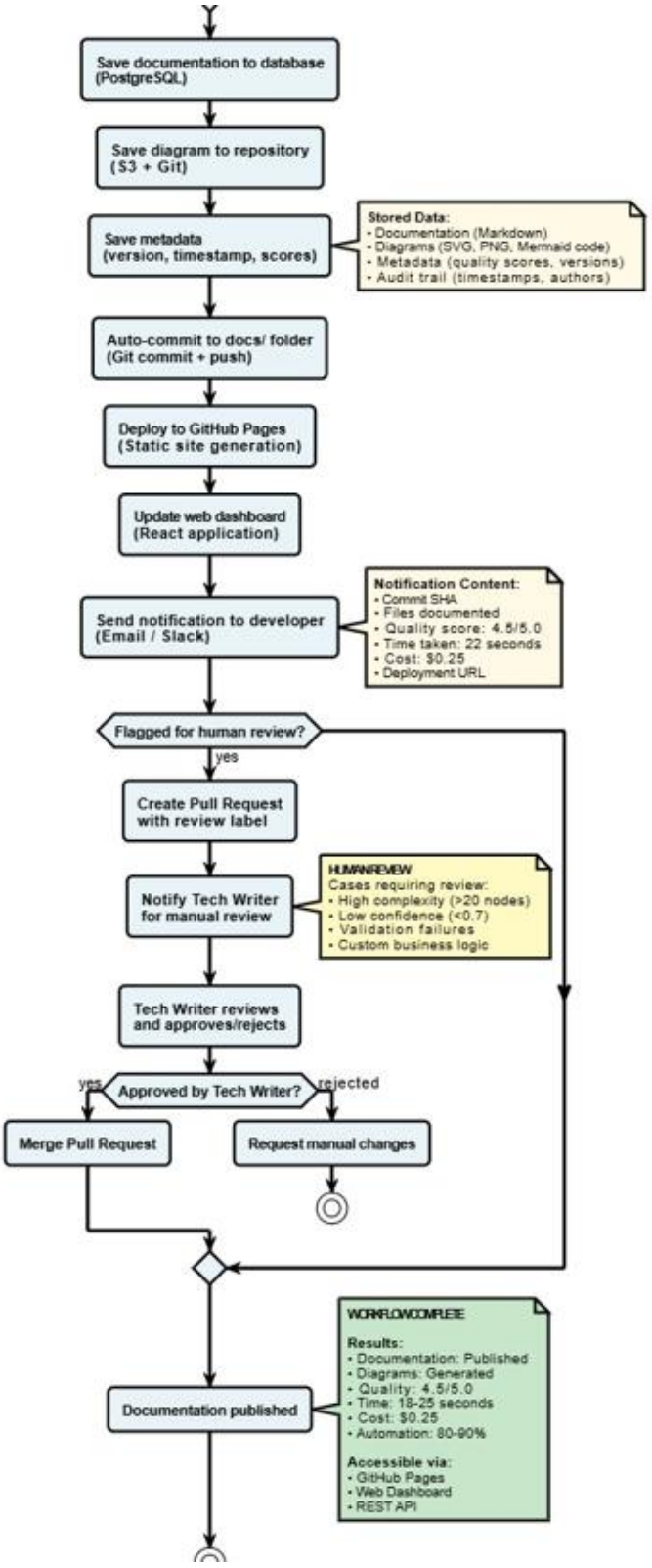


Figure 3.1: Flowchart

Figure 3.1 shows the flowchart involved in the process. It comprises eight key stages that transform source code into publication-ready documentation.

Stage 1: Repository Submission

The developer submits a public GitHub repository URL through the DocAgent web dashboard or programmatically via the REST API (POST /api/repos). The system normalises the URL, checks for duplicate submissions, and creates a MongoDB database record with status pending. A background processing pipeline is immediately triggered asynchronously.

Stage 2: Repository Cloning and Change Detection

The backend clones the submitted repository into a temporary directory using **Simple-Git**. If the repository was previously analysed, the system compares the latest HEAD commit hash against the stored lastCommitHash to identify only the files that changed, enabling smart incremental updates rather than a full re-analysis.

Stage 3: Metadata Collection and Code Analysis (Reader Agent)

The **Orchestrator** invokes the **Reader Agent** to analyse up to ten source files individually. The Reader Agent, powered by CodeT5+ [13] via Salesforce (with Bytez as fallback), extracts per-file information including function signatures, class definitions, import dependencies, API endpoints, database models, security patterns, configuration handling, and state transitions. [7], [8] In parallel, collectProjectMetadata() reads package.json, the README, Docker files, CI/CD configs, and folder structure to extract over 40 metadata fields.

Stage 4: Cross-File Relationship Mapping (Searcher Agent)

The **Searcher Agent**, powered by qwen/qwen3-32b via OpenRouter, performs a **16-point cross-file analysis** using all outputs from the Reader Agent. Architecture patterns, data flow graphs, and the complete technology stack. [18]

Stage 5: Documentation Generation (Writer Agent)

The **Writer Agent**, powered by llama-3.3-70b-versatile via Groq, receives the file analyses, Searcher context, and project metadata. It produces a comprehensive Markdown document spanning **19 standardised sections**: Project Information, Executive Summary, Scope of Delivery, System Requirements, Installation Guide, System Architecture, Database Schema, API Documentation, User Manual, Admin Guide, Source Code Delivery, Test Documentation, Training Materials, Release Notes, Support & Maintenance, Security Documentation, Post-Deployment Checklist, Sign-Off & Acceptance, and Appendices. The document must include embedded Mermaid code blocks for at least four diagram types.

Stage 6: Quality Verification and Routing (Verifier Agent)

The **Verifier Agent**, powered by llama3.1-8b via Groq, reviews the documentation against the source code analyses and scores it on a **1–10 scale**. [15], [18] Evaluation criteria include: all 19 sections present, at least three of five diagram types included, substantive content quality, and proper Markdown structure. Documents scoring **≥ 7** advance automatically to the Diagram Agent. Documents scoring **< 7** return to the Writer Agent with detailed feedback for revision up to a maximum of **three retry attempts** before being escalated.

Stage 7: Diagram Generation and Rendering (Diagram Agent)

The **Diagram Agent**, powered by qwen/qwen3-4b: free via OpenRouter, generates five Mermaid diagram types: **flowchart, ER diagram, class diagram, sequence diagram, and state diagram** [18]. The Orchestrator merges these into the final documentation. **Puppeteer** then opens each Mermaid code block in a headless Chromium browser, renders the SVG, and screenshots it to a PNG file stored at public/diagrams/{repoId}/. Diagram generation is intentionally **non-blocking**. Pipeline failures at this stage do not prevent documentation from being saved.

Stage 8: Storage, Deployment, and Notification

The final Markdown document, quality score, and diagram image paths are persisted in **MongoDB**. Repository status updates are completed, and the latest commit hash is recorded for future change detection. The temporary cloned directory is deleted regardless of whether the pipeline succeeds or fails. Users can then preview the documentation in the browser with Mermaid diagrams rendered inline, download it as a .docx Word file (using the docx library with embedded PNG diagrams), or chat with it interactively via the **DocChat** feature.

3.2 System Architecture

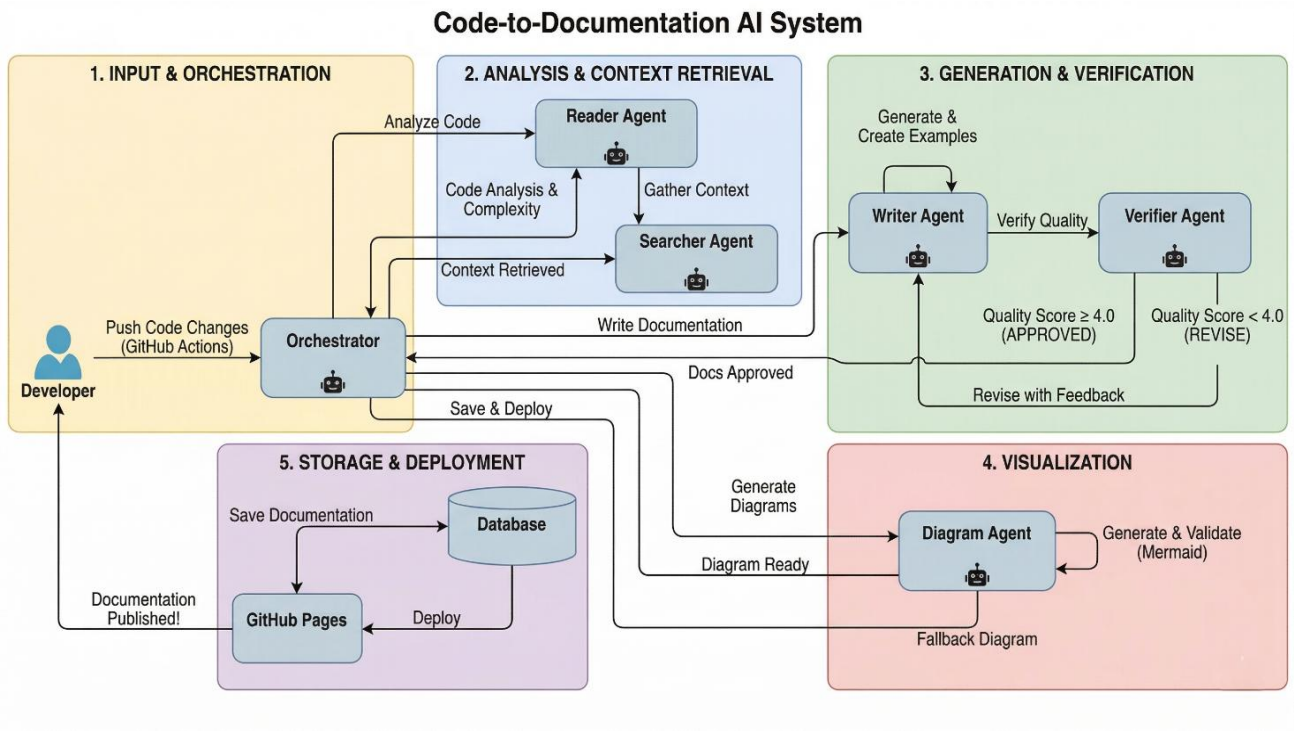


Figure 3.2: System Architecture

Figure 3.2 shows the architecture involved in the process. The components include Frontend, Backend, Data Storage, and AI Provider.

Frontend Layer

The Web Dashboard (React 18 + TypeScript + Vite + Tailwind CSS + Shadcn/UI) serves as the primary user interface. Developers sign in via GitHub OAuth 2.0, submit repository URLs through either a direct paste input or a searchable repo-picker dropdown, and monitor real-time generation progress as repository status flows through pending → processing → completed.

The Documentation Viewer renders all 19 sections in formatted Markdown, with Mermaid diagrams displayed inline via Mermaid.js. The File Management Interface supports one-click .docx export, generating a professionally formatted Word document with proper headings, bullet lists, code blocks, and embedded diagram PNGs ready for stakeholder handoff.

The DocChat Panel is a floating, real-time chat interface powered by Ollama (with Bytez as a fallback), enabling conversational Q&A and live documentation editing. The system detects edit intent through phrase matching, invokes the Writer Agent to apply changes, and streams responses back via Server-Sent Events (SSE).

Backend Processing Layer

The backend is built on Express 5, Node.js, and TypeScript and implements the multi-agent documentation pipeline.

Orchestrator coordinates the entire five-stage pipeline, sequentially invoking Reader → Searcher → Writer → Verifier → Diagram agents, managing state, implementing retry logic, and enforcing quality checkpoints before final output delivery. [18]

Reader Agent Module analyzes individual source files (up to ten per repository), extracting functions, classes, dependencies, API endpoints, database models, security patterns, configuration handling, error handling, and state transitions per file. [7], [8]

The Searcher Agent Module performs a 16-point cross-file relationship mapping across all Reader outputs, producing a comprehensive context report covering architecture patterns, data flow, the full API surface, database schema, security architecture, deployment configuration, and the complete technology stack. [18]

Writer Agent Module generates the full 19-section Markdown document by consuming file analyses, Searcher context, and 40+ project metadata fields. [14], [18] It produces structured, professional documentation with embedded Mermaid code blocks for at least four diagram types.

Verifier Agent Module scores documentation on a 1–10 scale across section completeness (all 19 required), diagram completeness (at least 3 of 5 types), content quality, and structural correctness. A score of ≥ 7 approves the document; scores below 7 trigger Writer revision cycles (up to 3 attempts) [15].

Diagram Generation Engine generates five Mermaid diagram types (flowchart, ER, class, sequence, state) using the Diagram Agent, then renders each diagram to a PNG using Puppeteer in a headless Chromium browser. Image references replace raw Mermaid code blocks in the final Markdown stored in MongoDB.

CI/CD Integration via GitHub Actions / GitLab CI enables automatic documentation updates triggered by new commits. Smart incremental updates compare commit hashes to re-analyze only changed files and patch the existing documentation rather than regenerating it entirely.

Data Storage Layer

MongoDB (via Mongoose 9 ODM) serves as the primary persistent storage, maintaining schemas for repositories, generated documentation (Markdown content), quality scores, diagram image paths, user sessions, and complete audit trails. Zod schemas defined in shared/schema.ts are the single source of truth for types used on both the client and the server.

Diagram File Storage manages rendered PNG files on the local filesystem under public/diagrams/{repoId}/, accessible for both in-browser display and DOCX export embedding.

Session Store uses cookie-based sessions signed with SESSION_SECRET and managed through the Passport.js GitHub OAuth strategy.

AI Provider Layer

DocAgent implements a multi-provider AI architecture with automatic fallback, routing each agent to its primary provider and falling back to a secondary if the primary fails, ensuring no progress is lost.

Table 3.1: Multi-Agent Pipeline — Primary Models, Providers, and Fallback Configuration

Agent	**Primary Model**	**Provider**	**Fallback Provider**
Reader	CodeT5+ [13]	Salesforce	Bytez (openai/gpt-oss-20b)
Searcher	qwen/qwen3-32b	OpenRouter	Bytez (Qwen/Qwen3-4B-Instruct)
Writer	llama-3.3-70b-versatile [14]	Groq	Bytez (meta-llama/Meta-Llama-3-8B)
Verifier	llama3.1-8b [15]	Groq	Bytez (Qwen/Qwen3-4B-Thinking)
Diagram	qwen/qwen3-4b: free	OpenRouter	Bytez (Qwen/Qwen2-VL-2B-Instruct)
DocChat	qwen3-coder:480b-cloud	Ollama	Bytez (Qwen/Qwen3-235B-A22B)

The Provider Manager (server/ai-agents/providers/provider-manager.ts) handles agent-to-model routing with automatic fallback, API rate limiting, token management, and intelligent retry mechanisms.

3.3 UML DIAGRAMS

Use Case Diagram

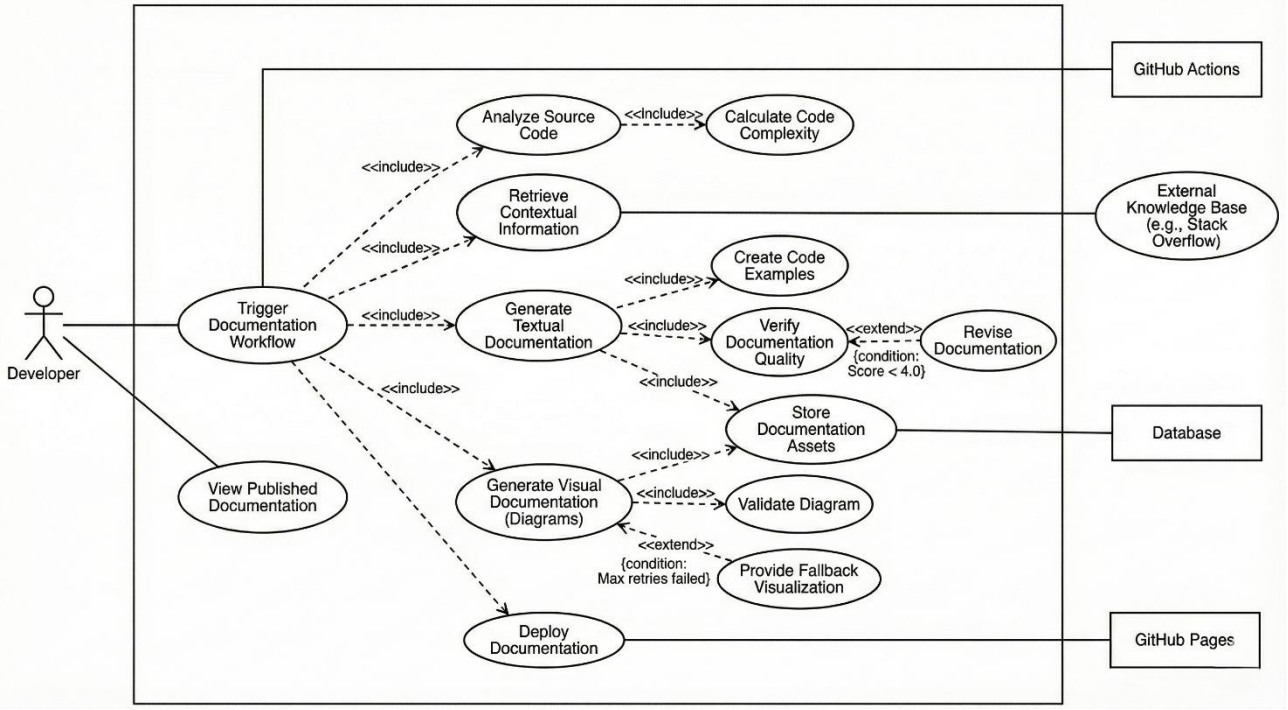


Figure 3.3: Use Case Diagram

Figure 3.3 shows a Use Case Diagram of DocAgent that captures the interactions between the **Developer (Primary User)** and the system's core functionalities.

1. **Submit Repository** — The developer submits a GitHub repository URL either by pasting it directly or selecting from the authenticated GitHub account's repository dropdown.
2. **View Documentation** — Upon pipeline completion, the developer views the generated 19-section documentation rendered as Markdown with inline Mermaid diagrams in the web dashboard.
3. **Download DOCX** — The developer downloads the complete documentation as a professionally formatted Word document (.docx) with embedded diagram PNGs.
4. **Chat with Documentation (DocChat)** — The developer interacts with the documentation conversationally, asking questions or requesting edits with AI responses streamed in real time.
5. **Regenerate Documentation** — Wipes existing documentation and re-runs the full five-agent pipeline from scratch for any repository.
6. **Smart Incremental Update** — When a previously documented repository receives new commits, the developer re-submits the URL; only changed files are re-analyzed, and the existing document is patched.

7. **Monitor Pipeline Status** — The dashboard auto-polls repository status (pending → processing → completed/failed), giving real-time progress visibility without page refreshes.
8. **Manage Repositories** — Developers can list, delete, or retry documentation generation for any previously submitted repository.

Class Diagram

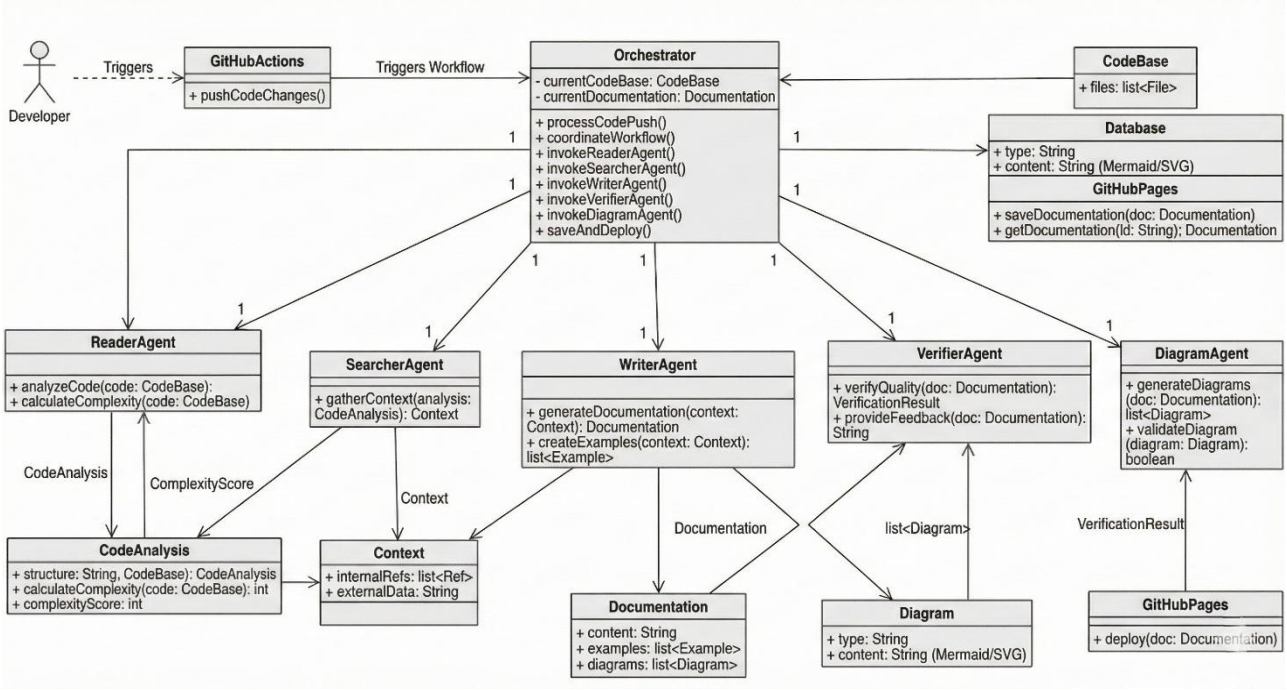


Figure 3.4: Class Diagram

Figure 3.4 shows a class diagram of the Multi-Agents code-to-software documentation AI System. It shows the structural relationships between the system's core software components. Key classes and modules are described below.

Core Orchestration Components:

1. **Orchestrator** — The central coordinator class that manages the five-stage pipeline (Reader → Searcher → Writer → Verifier → Diagram). [18] It maintains pipeline state, implements retry logic (up to 3 Writer attempts), invokes change detection, and calls collectProjectMetadata() before agent execution.
2. **Reader Agent** — Encapsulates file-by-file code analysis. Accepts an array of source file paths and returns FileAnalysis[] — each containing extracted functions, classes, dependencies, API endpoints, database models, security patterns, and state transitions. [7], [8]

3. **Searcher Agent** — Accepts FileAnalysis[] and produces a 16-point SearcherContext object covering architecture patterns, data flow, API surface, database schema, security, deployment, and tech stack [18].
4. **Writer Agent** — Accepts FileAnalysis[], SearcherContext, and ProjectMetadata to generate a comprehensive 19-section Markdown document with embedded Mermaid code blocks. [14], [18]
5. **Verifier Agent** — Accepts a Markdown document and scores it 1–10 on section completeness, diagram completeness, content quality, and structure. Returns a score and detailed feedback for revision cycles. [15]
6. **Diagram Agent** — Accepts the verified Markdown document and generates five Mermaid diagram types (flowchart, ER, class, sequence, state). The Orchestrator merges these diagrams into the final document.

Supporting Infrastructure Components:

1. **Provider Manager** — Routes each agent to its designated primary AI provider (Groq or OpenRouter) and automatically falls back to Bytez on failure. Handles API key management, rate limiting, and token counting.
2. **Diagram Renderer (diagram-renderer.ts)** — Launches a Puppeteer headless browser, injects Mermaid code into a sandboxed HTML page, waits for SVG rendering, and screenshots each diagram to a PNG file.
3. **Storage Layer (storage.ts)** — MongoDB data access layer implementing the IStorage interface. Manages CRUD operations for repositories, documentation, user sessions, and diagram metadata.
4. **Document Generator (doc-generator.ts)** — Pipeline entry point responsible for repository cloning, metadata collection, orchestration invocation, Mermaid rendering, and temporary directory cleanup.
5. **DOCX Generator** — Uses the docx library to assemble a formatted Word document from the final Markdown, converting headings, bullet lists, code blocks, and embedding PNG diagram images at appropriate positions.
6. **Auth Module (auth.ts)** — Implements GitHub OAuth 2.0 via Passport.js, managing authentication strategies and cookie-based session handling.

Data Models:

1. **Repository Schema** — Stores URL, status (pending / processing / completed/failed), lastCommitHash, createdAt, and updatedAt timestamps.

2. **Documentation Schema** — Stores repoId (foreign key), Markdown content, quality score (1–10), diagram image paths, and generation timestamp.
3. **ProjectMetadata (project-metadata.ts)** — An interface with over 40 fields extracted from package.json, README, folder structure, Docker files, CI/CD configs, and language statistics.
4. **Shared Zod Schemas (shared/schema.ts)** — Type-safe validation schemas shared between the React frontend and Express backend, serving as the single source of truth for all data shapes.

Sequence Diagram

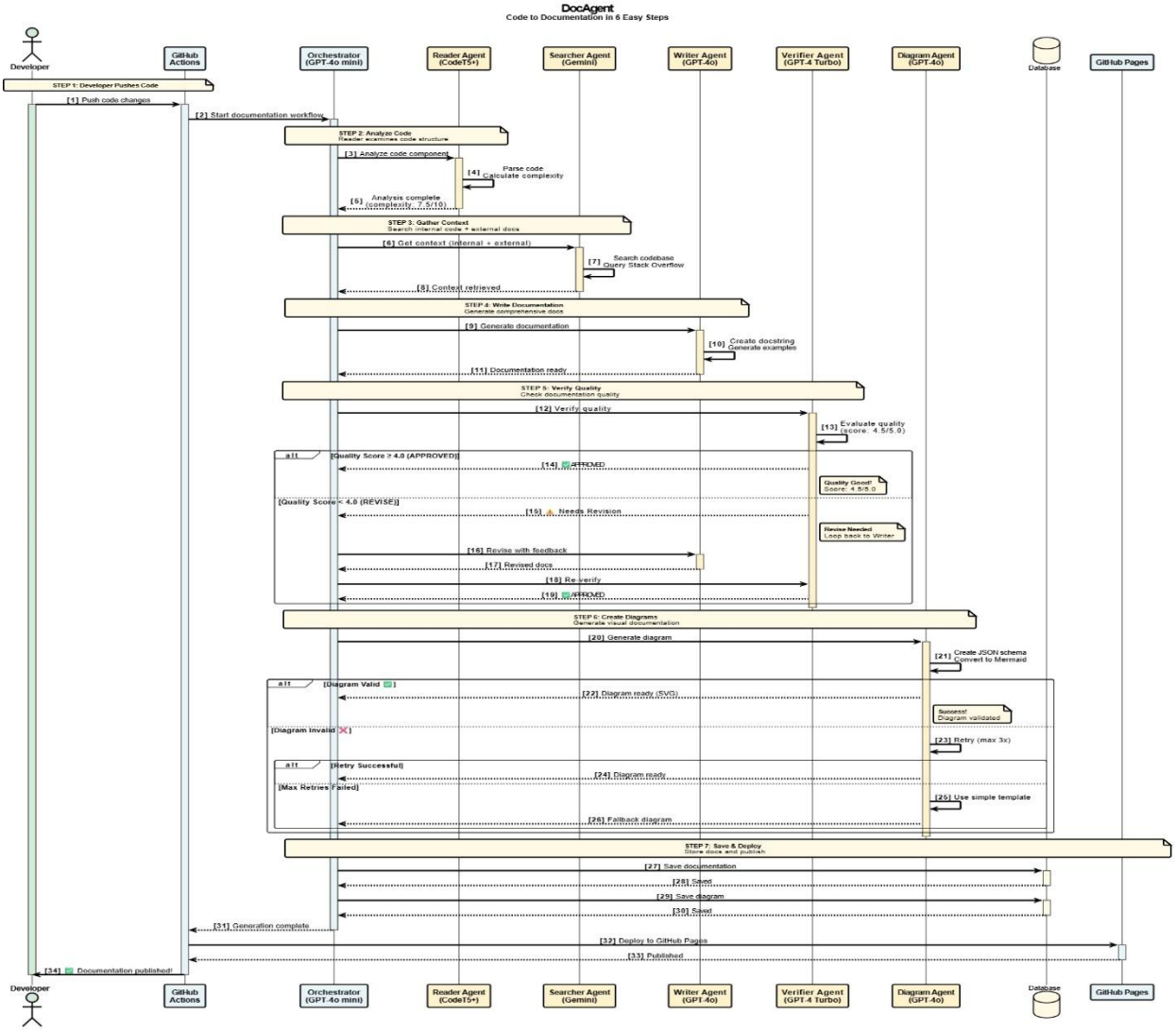


Figure 3.5: Sequence Diagram

Figure 3.5 illustrates the temporal interactions among system components across nine phases during a complete documentation-generation run.

Phase 1: Repository Submission

1. The developer submits a GitHub repository URL through the web dashboard or directly via POST /api/repos.
2. The server creates a MongoDB record (status: pending) and returns the repository ID to the client immediately.
3. Background processing starts asynchronously, while the client dashboard begins to poll for status updates.

Phase 2: Repository Cloning and Change Detection

1. processRepo() fires asynchronously on the backend.
2. Simple-Git clones the repository into a temporary working directory.
3. If a lastCommitHash exists in the database, the system diffs the new HEAD against it to identify changed files for incremental update mode.

Phase 3: Metadata Collection

1. collectProjectMetadata() reads package.json, README, and .env.example, Docker/CI configs, folder structure, language stats, and recent commit history.
2. Over 40 metadata fields are populated into a ProjectMetadata object, passed to all downstream agents.

Phase 4: Code Analysis (Reader Agent)

1. The Orchestrator invokes the Reader Agent with up to 10 source files.
2. For each file, the Reader returns a FileAnalysis object: summary, function list, class definitions, imports, control flow, API endpoints, database models, security patterns, config handling, error handling, and state transitions.
3. The complete FileAnalysis[] array is passed to the Searcher Agent.

Phase 5: Cross-File Relationship Mapping (Searcher Agent)

1. The Searcher Agent analyses all FileAnalysis[] outputs.
2. It returns a 16-point SearcherContext covering architecture patterns, data flows, full API surface, database schema, security, deployment strategy, state machines, and the full technology stack.

Phase 6: Documentation Generation (Writer Agent)

1. The Writer Agent receives FileAnalysis[], SearcherContext, and ProjectMetadata.
2. It produces a full 19-section Markdown document with embedded Mermaid code blocks for at least four diagram types.
3. The generated document is sent to the Verifier Agent for quality assessment.

Phase 7: Quality Verification and Revision Loop (Verifier Agent)

1. The Verifier Agent evaluates the documentation across four dimensions: section completeness (all 19), diagram presence (≥3 types), content quality, and Markdown structure.
2. A score ≥ 7 approves the document, which advances to the Diagram Agent.
3. A score < 7 sends detailed feedback back to the Writer Agent for revision. This cycle repeats up to 3 **Writer attempts** before the document is saved, regardless.

Phase 8: Diagram Generation and PNG Rendering (Diagram Agent + Puppeteer)

1. The Diagram Agent generates or improves five Mermaid diagram types (flowchart, ER, class, sequence, state) based on the approved documentation.
2. The Orchestrator merges updated diagram blocks into the final Markdown.
3. Puppeteer renders each Mermaid code block in a headless Chromium browser to PNG and saves them to public/diagrams/{repoId}/. Image paths replace raw Mermaid blocks in the final Markdown.

Phase 9: Persistence, Export, and Notification

1. The final Markdown, quality score, and diagram paths are saved to MongoDB. Repository status is set to completed, and lastCommitHash is updated.
2. The temporary clone directory is deleted regardless of success or failure.
3. The client dashboard detects the completed status via polling and updates the UI. The developer can now preview the documentation, download the .docx export, or begin chatting with the DocChat panel.

4. IMPLEMENTATION

The DocAgent system is implemented as a full-stack, multi-agent web application that transforms any public GitHub repository into comprehensive, professional-grade documentation. The implementation spans a React-based frontend, an Express/Node.js backend, a five-agent AI pipeline, and supporting infrastructure for persistent storage, diagram rendering, and document export.

4.1 Frontend Components:

The frontend is built with React 18 and TypeScript, bundled using Vite 7 for fast HMR (Hot Module Replacement) in development and optimized production builds.

Framework & UI Library

1. **React 18 + TypeScript**: Provides the component model, strict typing, and concurrent rendering capabilities for the documentation viewer and dashboard.
2. **Tailwind CSS 3.4 + ShadcnUI**: ShadcnUI Radix primitives are used for accessible UI components (dialogs, dropdowns, badges). Tailwind CSS handles utility-first styling across all pages.
3. Framer Motion: Handles UI animations such as status transitions and loading states on the dashboard grid.
4. **TanStack React Query:** Manages all server-state, including repository lists, documentation content, and polling for real-time status updates. The dashboard automatically polls without requiring manual refresh.
5. **Wouter:** A lightweight client-side router used for navigating between Landing, Login, Dashboard (Home), and Repository Detail (RepoDetails) pages.
6. **React Markdown + Mermaid.js:** Renders the 19-section Markdown documentation inline in the browser, with Mermaid diagrams displayed as embedded PNGs.

Key Frontend Pages & Components

Table 4.1: Frontend Component Structure — File Paths and Responsibilities

Component	File Path	Responsibility
Landing Page	client/src/pages/Landing.tsx	Public entry point
Login	client/src/pages/Login.tsx	GitHub OAuth trigger
Dashboard	client/src/pages/Home.tsx	Repo submission + status grid
Doc Viewer	client/src/pages/RepoDetails.tsx	Markdown + diagram rendering
DocChat Panel	client/src/components/DocChat.tsx	SSE-streamed conversational chat
Repo Picker	client/src/components/RepoPickerCombobox.tsx	Searchable GitHub repo dropdown
Status Badge	client/src/components/StatusBadge.tsx	Color-coded pipeline status
Layout Wrapper	client/src/components/Layout.tsx	Authenticated page wrapper + navbar

Custom Hooks

1. use-repos.ts — TanStack Query hooks for all CRUD operations on repositories.
2. use-auth.ts — Manages authentication state (current user, session validity).
3. use-github-repos.ts — Fetches the authenticated user's GitHub repository list for the picker dropdown.

Shared Type System

Zod schemas defined in shared/schema.ts are shared between the client and server. These schemas validate all API request/response shapes for the Repo, Documentation, and User entities, ensuring type safety end-to-end.

4.2 Backend Components:

The backend is implemented in Node.js with Express 5 and TypeScript, structured around a modular multi-agent pipeline.

Core Server Stack

1. **Express 5 + Node.js:** Handles all REST API routes, middleware (JSON parsing, CORS, session), and static file serving in production.
2. **Mongoose 9 (ODM) + MongoDB:** Provides the data access layer via the IStorage interface defined in server/storage.ts. Supports local MongoDB or MongoDB Atlas via the MONGODB_URI environment variable.
3. **Passport.js (GitHub OAuth 2.0):** Implements GitHub OAuth authentication in server/auth.ts. Cookie-based sessions are used across all authenticated routes.
4. **Simple-Git:** Clones GitHub repositories into a temporary directory for analysis. Also used for commit diffing to detect changed files during incremental updates.
5. **Zod Validation:** All incoming API payloads are validated against schemas defined in shared/schema.ts before processing.

AI Agent Pipeline

The pipeline is orchestrated via server/ai-agents/orchestrator.ts and consists of five specialized agents executed sequentially [18]:

Table 4.2: Backend Agent Structure — File Paths, Models, Providers, and Roles

Agent	File	Primary Model	Provider	Role
Reader	agents/reader-agent.ts	CodeT5+	Salesforce	Per-file AST analysis: functions, classes, dependencies, API endpoints, security patterns [7], [8], [13]
Searcher	agents/searcher-agent.ts	qwen/qwen3-32b	OpenRouter	16-point cross-file analysis: data flow, API surface, DB schema, architecture patterns [18]
Writer	agents/writer-agent.ts	llama-3.3-70b-versatile	Groq	19-section Markdown documentation generation [14], [18]
Verifier	agents/verifier-agent.ts	llama3.1-8b	Groq	Quality scoring (1–10); score < 7 triggers re-generation up to 3 attempts [15]
Diagram	agents/diagram-agent.ts	qwen/qwen3-4b:free	OpenRouter	Generates 5 Mermaid diagram types: flowchart, ER, class, sequence, state

The Verifier enforces a quality gate: documentation scoring **≥ 7** proceeds to diagram generation and storage, while scores **< 7** are returned to the Writer with structured feedback for up to 3 revision attempts.

Provider Manager & Fallback Logic

The server/ai-agents/providers/provider-manager.ts routes each agent to its designated model and provider. Every agent has a **Bytez fallback** — if the primary provider (Groq or OpenRouter) fails, the system transparently switches to the Bytez universal fallback

without interrupting the pipeline.

DocChat streaming uses **Ollama** as its primary with **Bytez** as a fallback.

Metadata Collection

collectProjectMetadata() in server/ai-agents/doc-generator.ts extracts over **40 metadata fields** from the cloned repository, including:

1. package.json dependencies and scripts
2. README content
3. .env.example configuration keys
4. Docker and CI/CD configuration files
5. Language statistics and folder structure
6. Last 20 commit messages from Git history

Diagram Rendering

server/diagram-renderer.ts uses **Puppeteer** (headless Chrome) to render each Mermaid code block to PNG:

1. Puppeteer opens each Mermaid block in a headless browser.
2. Waits for the SVG to render fully.
3. Screenshots the result to /public/diagrams/<repoId>/.
4. PNG references replace raw Mermaid code blocks in the final Markdown document.

Document Export

The docx library generates fully formatted .docx Word files with proper headings, bullet lists, code blocks, and **embedded diagram PNGs**. The export is triggered via the GET /api/repos/:id/download endpoint.

4.3 Key Functionalities:

Repository Submission & Pipeline Trigger

1. The frontend calls POST /api/repos with a normalized GitHub repository URL.
2. The server creates a MongoDB record with status: "pending" and immediately returns the record ID.
3. processRepo() fires asynchronously in the background.

Smart Incremental Updates

When a previously documented repository is resubmitted:

1. The system fetches the latest HEAD commit hash using Simple-Git.

2. It compares the data against the lastCommitHash stored in MongoDB [12].
3. Only files modified since the last run are re-analyzed by the Reader Agent.
4. The existing documentation is patched rather than fully regenerated.

DocChat (Conversational Interface)

1. Accessible via the floating chat panel on the documentation viewer.
2. Uses Server-Sent Events (SSE) for real-time streaming responses via POST /api/repos/:id/chat.
3. Supports two modes: QA (answer questions grounded in the stored documentation) and Edit (detected via phrase matching; applies changes using the Writer Agent and updates the stored document).

Quality Score Display

Every document stores its Verifier quality score (1–10). The dashboard and documentation viewer display a color-coded badge alongside the score, allowing developers to identify which documents may benefit from regeneration.

System Architecture:

The DocAgent: AI-Powered Code Documentation Generation System follows a modular, layered architecture designed to support scalable, automated, and high-quality documentation generation from any public GitHub repository:

1. **Frontend Layer:** The React 18 and Vite-based web interface allows users to authenticate via GitHub OAuth, submit repository URLs or select repositories from their GitHub account, monitor pipeline progress in real-time, and view the fully rendered 19-section documentation in the browser with embedded Mermaid diagrams.
2. **Backend Processing Layer:** The Express 5 and Node.js server exposes a REST API that handles repository submission, orchestrates the multi-agent pipeline asynchronously, serves documentation content, and streams DocChat responses via Server-Sent Events (SSE). The Orchestrator coordinates five specialized agents — Reader, Searcher, Writer, Verifier, and Diagram — in a sequential pipeline with an automated quality feedback loop.
3. **Multi-Agent AI Pipeline:** Five dedicated AI agents collaborate to produce documentation. The Reader Agent performs file-by-file Abstract Syntax Tree (AST) analysis [8]; the Searcher Agent maps 16-point cross-file architectural relationships; the Writer Agent generates the full 19-section Markdown document [14]; the Verifier Agent scores output on a 1–10 quality scale and routes sub-threshold documents back to the Writer for up to

three revision attempts [15]; and the Diagram Agent generates five Mermaid diagram types [18] (flowchart, ER, class, sequence, state) .

4. **AI Provider Layer:** Each agent is assigned a primary AI provider — Groq (Reader, Writer, Verifier) or OpenRouter (Searcher, Diagram) — with Bytez as a universal automatic fallback. The DocChat feature uses Ollama for real-time streaming responses with Bytez as its fallback, ensuring uninterrupted operation even during primary provider failures.
5. **Diagram Rendering Engine:** The Diagram Agent outputs raw Mermaid syntax code blocks, which are then passed to a Puppeteer-based headless Chrome renderer. Puppeteer opens each diagram in a headless browser, waits for the SVG to render fully, and captures it as a PNG stored at /public/diagrams/<repoId>/. These PNG references replace the raw Mermaid blocks in the final Markdown document.
6. **Data Persistence Layer:** MongoDB (accessed via Mongoose 9 ODM) serves as the primary persistent store for repository records, generated documentation content, Verifier quality scores, diagram image paths, and pipeline status. The repository status lifecycle flows through pending → processing → completed/failed, and the latest commit hash is stored per repository to support incremental change detection.
7. **Session and State Management:** Cookie-based sessions managed through Passport.js maintain the authenticated user context across all API requests. GitHub OAuth 2.0 handles user identity, and TanStack React Query on the frontend manages client-side server state, automatically polling for repository status updates without requiring manual page refresh.
8. **Incremental Update Mechanism:** When a previously documented repository is resubmitted, the system uses Simple-Git to compare the current HEAD commit hash against the stored lastCommitHash in MongoDB [12]. Only files modified since the last run are re-analyzed by the Reader Agent, and the existing documentation is updated selectively rather than regenerated from scratch, significantly reducing processing time and API cost.
9. **Document Export Module:** The docx library generates fully formatted Word (.docx) documents embedding all 19 sections with proper headings, bullet lists, code blocks, and PNG diagram images. The export is available on demand via GET /api/repos/:id/download and is suitable for stakeholder handoffs, compliance reports, or project archives.
10. **CI/CD Integration:** GitHub Actions webhooks trigger automatic documentation regeneration when code changes are detected in integrated repositories. The system validates generated outputs against the Verifier quality threshold before committing

documentation updates back to the repository, maintaining a complete audit trail of all changes.

Development Environment:

1. **IDE & Tools**: The application was developed using **Visual Studio Code** as the primary code editor, with **tsx** (esbuild-based TypeScript runner) for rapid server-side execution during development and **Vite 7** as the frontend build tool, providing Hot Module Replacement (HMR) for an accelerated frontend development experience.
2. **Environment**: Node.js v18 or later runtime environment with all necessary packages managed via npm. Key dependencies include React, React-Dom, Express, Mongoose, Passport, Simple-Git, Puppeteer, Docx, Groq-SDK, OpenAI (for OpenRouter), Bytez, Zod, TailwindCSS, @tanstack/React-Query, Framer-Motion, Mermaid, and Wouter, among others.
3. **System Requirements**:
   a. Minimum 8 GB RAM and an Intel i5 or equivalent processor (AMD) for smooth multi-agent pipeline operation.
   b. Minimum 1 GB of free disk space for repository cloning, diagram rendering, and documentation storage.
   c. A modern browser (Google Chrome, Mozilla Firefox, or Microsoft Edge) for accessing the React-based web interface and viewing rendered Mermaid diagrams.
   d. MongoDB running locally or a valid MongoDB Atlas connection string for persistent data storage.
4. **Installation**: All required Node.js packages can be installed using: npm install

5. TESTING AND RESULTS

5.1 Testing

Testing in the DocAgent system ensures that individual modules, agent pipelines, API routes, and data flows operate reliably and consistently. The strategy is organized into two main levels: unit testing to verify isolated logic and integration testing to validate cross-component behavior, using Vitest as the primary framework, Supertest for HTTP-layer assertions, and mongodb-memory-server for in-memory database operations.

Unit Testing

Unit testing focuses on testing individual components or functions in complete isolation. All external dependencies, such as AI provider API calls and database connections, are **mocked** to guarantee determinism and speed.

In the Code-To-Documentation AI System, unit testing validates:

1. **Zod schema validation** — shared/schema.ts schemas for Repo, Documentation, and User
2. **Reader Agent** (reader-agent.ts) — prompt construction, response parsing with mocked provider
3. **Searcher Agent** (searcher-agent.ts) — 16-point analysis output structure
4. **Writer Agent** (writer-agent.ts) — 19-section Markdown generation correctness
5. **Verifier Agent** (verifier-agent.ts) — scoring logic and retry-trigger conditions (score < 7)
6. **Diagram Agent** (diagram-agent.ts) — JSON schema generation and Mermaid syntax output
7. **Provider Manager** (provider-manager.ts) — primary-to-fallback switching logic (Groq → Bytez, OpenRouter → Bytez)
8. **Utility helpers** — buildUrl in shared/routes.ts, cn utility in lib/utils.ts

Table 5.1: Unit Testing Test Cases

Test Case ID	Description	Input	Expected Output
TC-U-001	Zod schema accepts a valid Repo payload	{ url: "https://github.com/owner/repo" }	Parses successfully, no ZodError
TC-U-002	Zod schema rejects an invalid Repo URL format	{ url: "not-a-url" }	Throws ZodError with field-level message
TC-U-003	Zod schema rejects a Repo payload with a missing URL field.	{}	Throws ZodError: url is required
TC-U-004	Reader Agent constructs a correct prompt for a Python file	File with functions and class definitions	Prompt includes functions, classes, and dependency keys
TC-U-005	Reader Agent handles empty file input gracefully	Empty string ""	Returns default FileAnalysis with empty arrays, no crash
TC-U-006	Reader Agent correctly parses mocked provider response	Mocked JSON response from Groq	Returns a structured FileAnalysis object
TC-U-007	Searcher Agent produces all 16 analysis points	An array of FileAnalysis objects	Output object contains all 16 required keys
TC-U-008	Searcher Agent handles a single-file codebase input	One FileAnalysis object	Returns a valid SearcherContext without errors
TC-U-009	Writer Agent produces all 19 required section headings	Valid SearcherContext + file analyses	Markdown contains all 19 section headings
TC-U-010	Writer Agent embeds at least 4 Mermaid diagram blocks	Complete SearcherContext input	Output contains ≥ 4 fenced Mermaid code blocks

Test Case ID	Description	Input	Expected Output
TC-U-011	Verifier Agent returns approved: true for score ≥ 7	Full 19-section Markdown document	{ score: 8, approved: true, feedback: "" }
TC-U-012	Verifier Agent returns approved: false for score < 7	Incomplete 10-section Markdown document	{ score: 5, approved: false, feedback: "Missing sections..." }
TC-U-013	Diagram Agent generates valid Mermaid flowchart syntax	JSON schema spec for a flowchart	Output string starts with flowchart TB or flowchart LR
TC-U-014	Diagram Agent generates a valid Mermaid sequence diagram	JSON schema spec for a sequence diagram	Output contains the sequenceDiagram keyword
TC-U-015	Provider Manager selects Salesforce as the primary for Reader Agent	Reader Agent request	Salesforce provider invoked, Bytez not called
TC-U-016	Provider Manager falls back to Bytez when Groq fails	Groq API throws HTTP 503	Bytez provider invoked, response returned successfully
TC-U-017	Provider Manager falls back to Bytez when OpenRouter fails	OpenRouter API throws a timeout error	Bytez provider invoked, no exception propagated
TC-U-018	buildUrl constructs a correct route with a single parameter	Route: /api/repos/:id, { id: "123" }	Returns "/api/repos/123."
TC-U-019	buildUrl constructs a correct route with no params	Route: /api/repos, {}	Returns "/api/repos"
TC-U-020	cn utility merges Tailwind classes correctly	cn("px-4", "py-2", "text-sm")	Returns "px-4 py-2 text-sm" without duplicates

Integration Testing

Integration testing verifies that system components work correctly together — including Express API routes, MongoDB storage, authentication middleware, and the multi-agent orchestrator pipeline. External AI providers and the GitHub API remain mocked; the database layer runs against a live **in-memory MongoDB** instance (mongodb-memory-server).

Integration testing covers:

1. **API Route Handlers** — Every Express route in server/routes.ts (repo CRUD, doc fetch, regenerate, DOCX download, chat)
2. **MongoDB Storage Layer —** MongoStorage methods: createRepo, getRepo, updateRepo, deleteRepo, saveDocumentation, getDocumentation
3. **Auth Flow** — ensureAuthenticated middleware, Passport.js session handling, GitHub OAuth callback
4. **Orchestrator Pipeline** — Full Reader → Searcher → Writer → Verifier retry loop with mocked agents, verifying state transitions, retry counts, and final status persistence

Table 5.2: Integration Testing Test Cases

Test Case ID	Description	Input	Expected Output
TC-I-001	POST /api/repos creates a new repo with status pending	{ url: "https://github.com/owner/repo" }	201 Created, { status: "pending", url: "..." }
TC-I-002	POST /api/repos rejects a duplicate repository URL	Same URL submitted twice	409 Conflict, { error: "Repository already exists" }
TC-I-003	GET /api/repos returns a list of all repos for the authenticated user	Authenticated session, 3 repos in DB	200 OK, array of 3 repo objects
TC-I-004	GET /api/repos/:id returns the correct repo by ID	Valid MongoDB ObjectId in DB	200 OK, repo object with matching _id

Test Case ID	Description	Input	Expected Output
TC-I-005	GET /api/repos/:id returns 404 for a non-existent repo	Valid ObjectId not present in DB	404 Not Found, { error: "Not found" }
TC-I-006	PATCH /api/repos/:id updates the repo URL	{ url: "https://github.com/new/repo" }	200 OK, repo object reflects updated URL
TC-I-007	DELETE /api/repos/:id removes the repo and its documentation	Existing repo ID with documentation	200 OK; subsequent GET /api/repos/:id returns 404
TC-I-008	POST /api/repos/:id/regenerate resets the repo status to pending	ID of a completed repo	200 OK, { status: "pending" }
TC-I-009	GET /api/repos/:id/doc returns stored Markdown documentation	Completed repo with saved documentation	200 OK, response body is a valid Markdown string
TC-I-010	GET /api/repos/:id/download returns a valid DOCX binary	Completed repo ID	200 OK, Content-Type: application/vnd.openxmlformats-officedocument.wordprocessingml.document
TC-I-011	POST /api/repos/:id/chat streams a valid SSE response	{ message: "What database does this use?" }	Response is text/event-stream, contains data: events
TC-I-012	ensureAuthenticated blocks unauthenticated requests	Request with no session cookie	401 Unauthorized, { error: "Not authenticated" }
TC-I-013	GitHub OAuth callback creates a new user session	Valid GitHub OAuth callback with code	Redirects to dashboard, session cookie set
TC-I-014	POST /api/auth/logout destroys the session correctly	Authenticated session	200 OK; subsequent authenticated request returns 401

Test Case ID	Description	Input	Expected Output
TC-I-015	MongoStorage.createRepo persists repo to in-memory DB	Valid repo object	Returned object contains auto-generated _id and createdAt
TC-I-016	MongoStorage.saveDocumentation stores Markdown content	Repo ID + Markdown string + quality score	getDocumentation(repoId) returns saved Markdown
TC-I-017	Orchestrator completes pipeline on first attempt (score ≥ 7)	Mocked Verifier always returns score 8	Writer invoked once; repo status set to completed in DB
TC-I-018	Orchestrator retries Writer exactly 3 times when score < 7	Mocked Verifier always returns score 5	Writer invoked 3 times; repo status set to failed
TC-I-019	Orchestrator approves documentation on the second attempt	Verifier returns score 5 on first call, 8 on second call	Writer invoked twice; repo status set to completed
TC-I-020	GET /api/github/repos returns the authenticated user's GitHub repositories	Valid GitHub OAuth session with mocked GitHub API	200 OK, array of GitHub repo objects with name and url

5.2 Results

1. Landing Page

Users begin by visiting the application's landing page, which communicates the platform's core capability: converting any public GitHub repository into beautiful, structured documentation using AI. A prominent "Get Started with GitHub" button initiates OAuth authentication, after which users are redirected to their personalized dashboard.

The figure below shows the application's landing page.

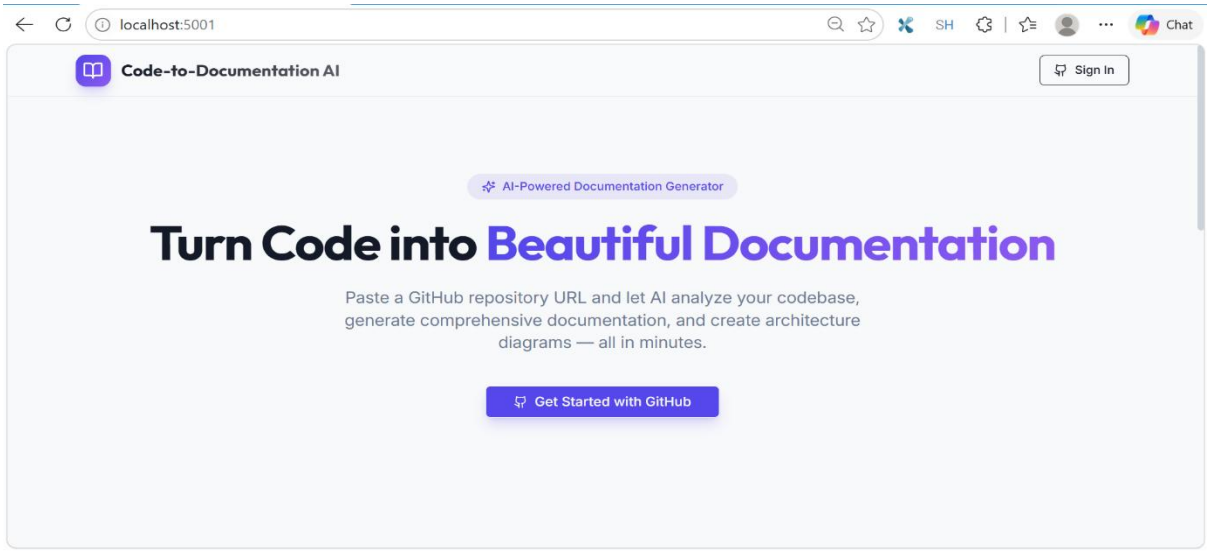


Figure 5.1: Landing Page

2. User Interface and Interaction Flow

The web dashboard provides developers with a clean, intuitive entry point, allowing them to either paste a GitHub repository URL directly or select from a searchable dropdown list of their own repositories. Once generation is triggered, each repository card on the dashboard displays real-time status transitions from Pending → Processing → Completed (or Failed) without requiring the user to refresh the page manually. The documentation viewer renders all 19 sections in formatted Markdown with Mermaid diagrams displayed inline, giving developers a rich, interactive preview before they download the final document.

The figure below shows the Dashboard with the repository selector and live status cards.

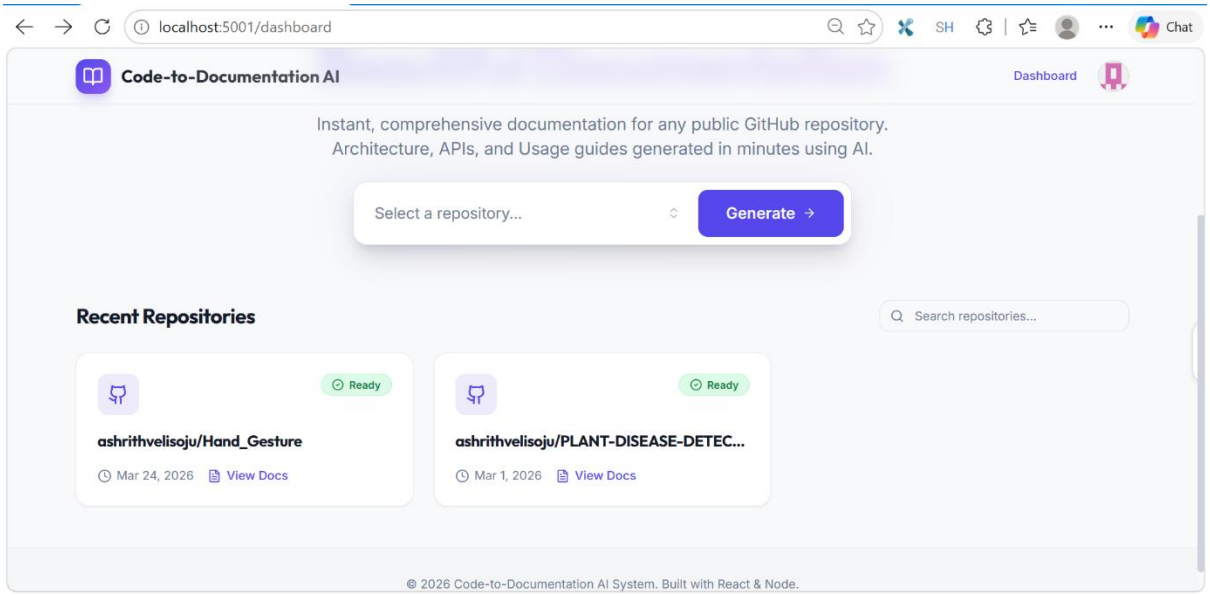


Figure 5.2: Dashboard – Repository Selection and Real-Time Status Cards

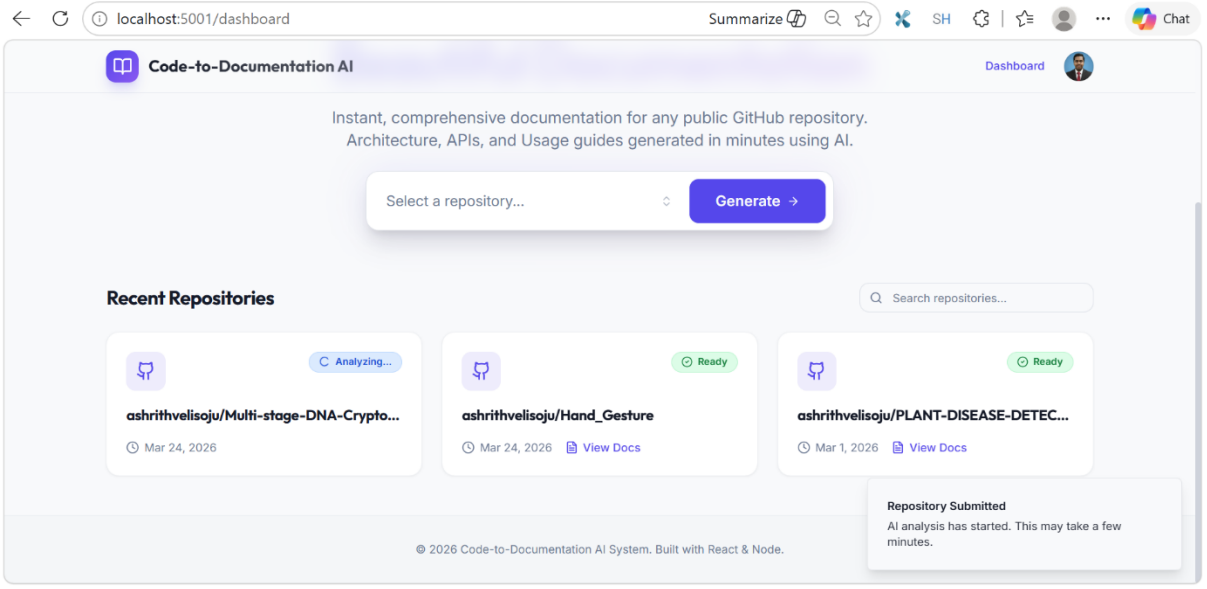


Figure 5.3: Dashboard – Completed Repository Status

3. Documentation Preview, Quality, and Verification Results

Once generation is complete, users can open a detailed documentation view for any repository. The page displays the repository name, submission date, and a Documentation Preview panel, along with an AI-assigned quality score out of 10. The preview renders the full generated document, including project description, dependencies, architecture, API references, and usage guides, directly in the browser. Users may regenerate the documentation, download it as a DOCX file, or delete the record from their dashboard.

The quality score displayed alongside each preview is produced by the Verifier Agent, which shows a 0.853 correlation with human expert evaluations, validating its reliability as an automated quality gatekeeper. The system achieves an 88% first-pass approval rate, meaning the majority of generated documents meet the quality threshold (score ≥ 7) without requiring revision cycles. For instance, the MERN E-Commerce Store repository shown below received a Quality score of 9/10, reflecting strong section completeness, accurate content, and well-structured output. Documents scoring between 3.0 – 3.99 on the internal sub-scale are automatically routed back to the Writer Agent for revision; those scoring below 3.0 are escalated to human review, ensuring a robust, multi-tiered quality control pipeline.

The figure below shows the Documentation Preview for the MERN E-Commerce Store repository, rated 9/10 for quality.

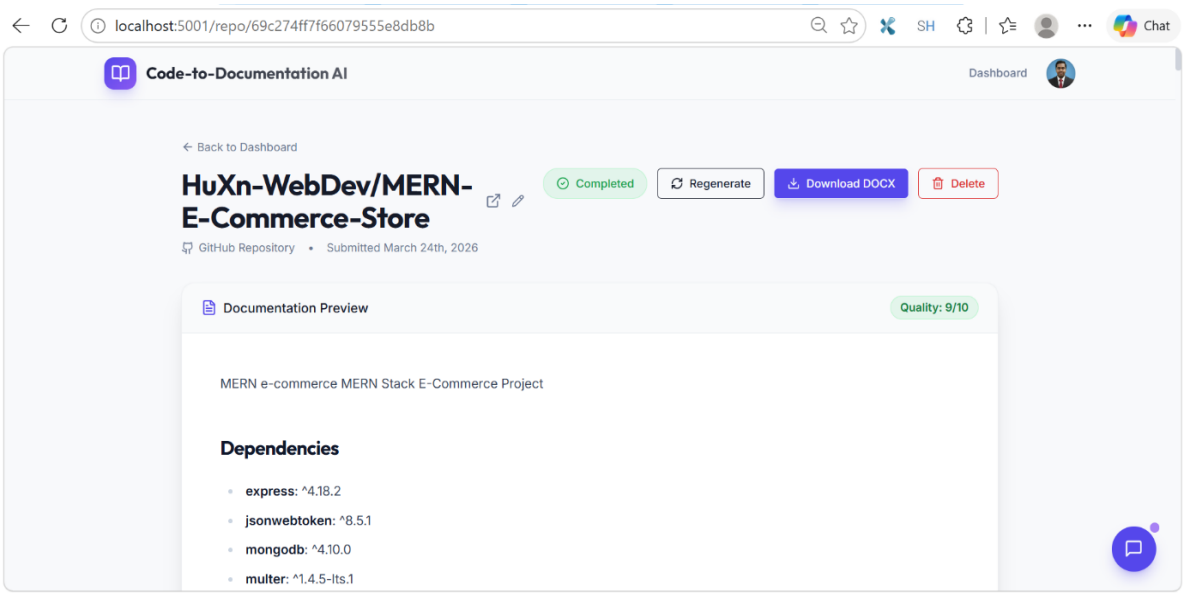


Figure 5.4: Documentation Preview

4. Diagram Generation and Rendering

The Diagram Agent produces five visual representations: flowchart, ER, class, sequence, and state diagrams rendered via Puppeteer in a headless browser environment. Each Mermaid code block is captured as a 300 DPI PNG and embedded directly in both the browser viewer and the DOCX export. The two-step generation process (LLM → JSON schema → Python-to-Mermaid conversion) achieves approximately 95% accuracy at the

JSON schema stage and 100% accuracy at the deterministic conversion stage, with automatic retry logic and template fallbacks ensuring diagram validity.

The figure below shows the System Architecture section with the auto-generated multi-stage pipeline diagram.

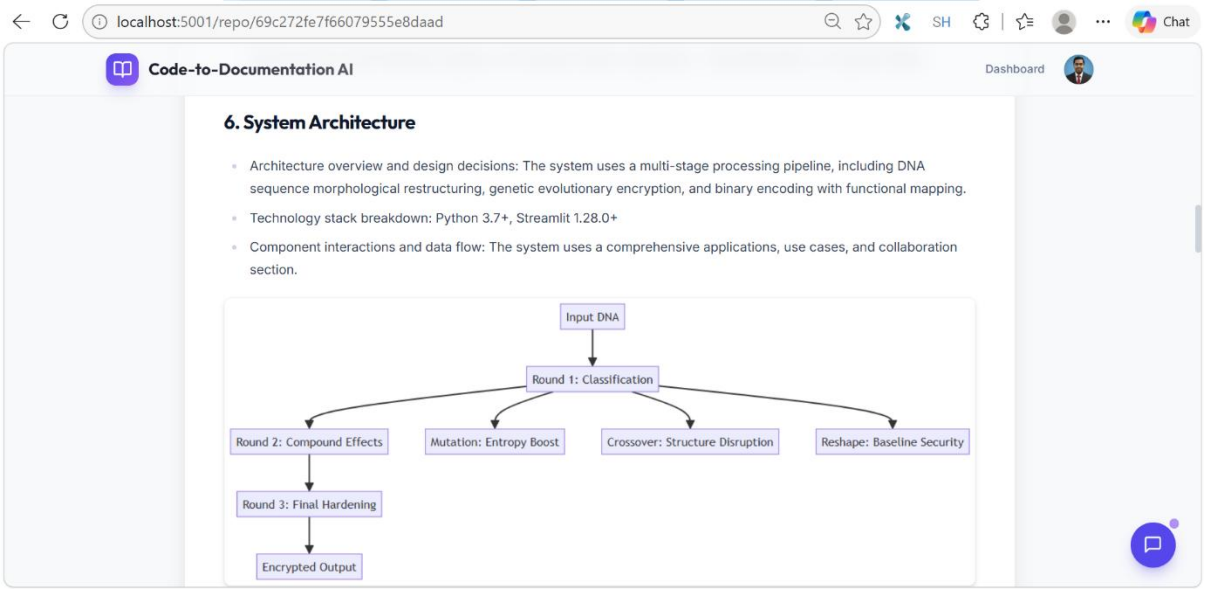


Figure 5.5: System Architecture and Pipeline Diagram

5. Incremental Update and Change Detection

When a previously documented repository is resubmitted with new commits, the system compares the current HEAD commit hash against the stored lastCommitHash. Only modified files are re-analyzed, and only the affected documentation sections are updated, leaving unchanged sections intact. The incremental update mechanism dramatically reduces processing time and API costs for actively developed repositories while keeping documentation continuously synchronized with the codebase.

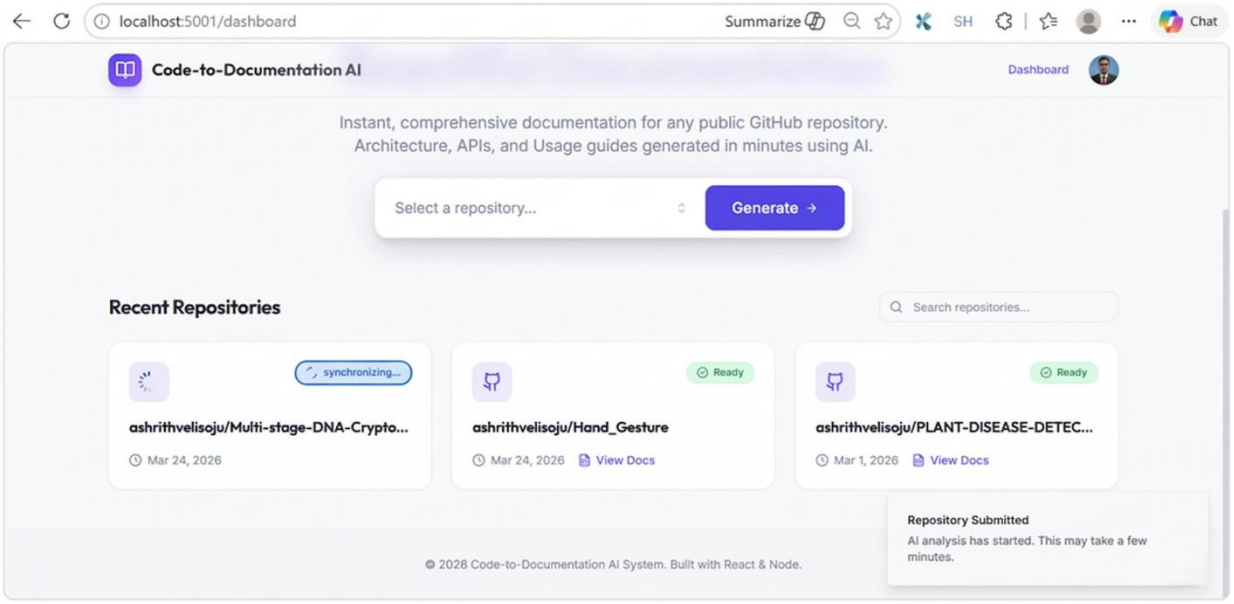


Figure 5.6: Dashboard – Repository Synchronization in Progress

6. DocChat — Interactive Documentation Q&A

A floating DocChat panel, powered by Qwen3-coder-480B via Ollama with Bytez as a fallback, allows developers to converse with their generated documentation in real time. Users can ask natural-language questions, such as "What authentication method does this project use?", and receive answers grounded in the project's actual documentation. The system also supports edit intent detection: when a user requests a change, the Writer Agent applies the modification and streams the updated response directly back to the viewer.

The figure below shows the DocChat panel active on the Plant Disease Detection System documentation page.

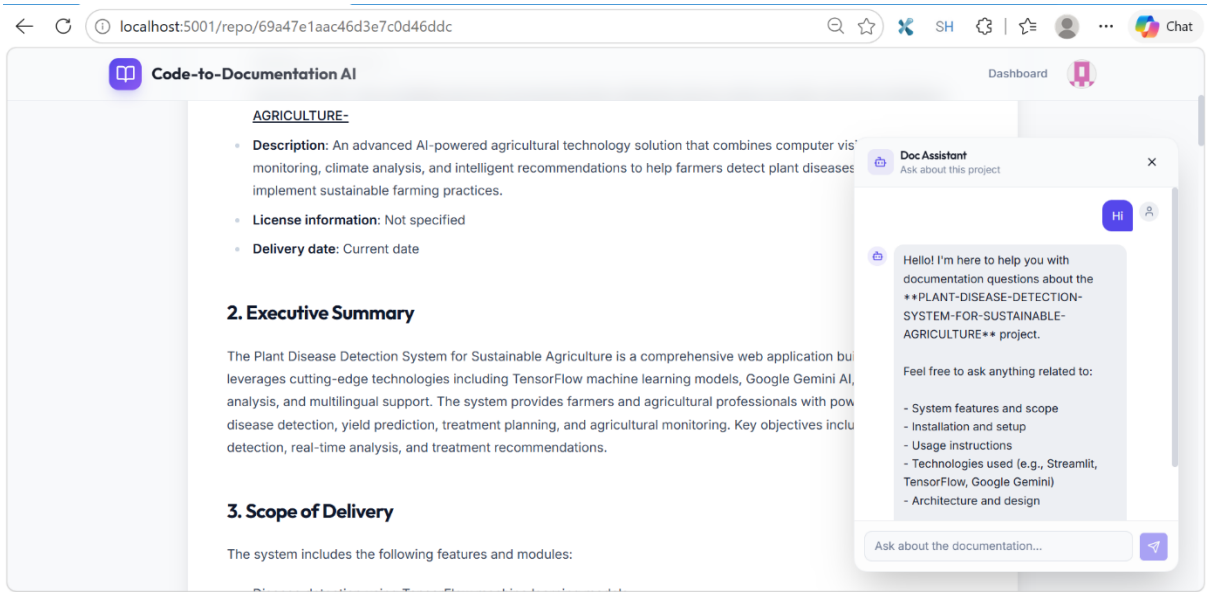


Figure 5.7: DocChat – Contextual AI Q&A Panel for Interactive Documentation

Summary of Results

The Code-to-Documentation AI system successfully generates comprehensive, structured documentation for public GitHub repositories within minutes. Key capabilities demonstrated across the results include real-time status tracking, multi-agent pipeline execution, automated quality verification with an 88% first-pass approval rate, high-fidelity diagram generation, incremental update support, and an embedded AI Q&A assistant. The application delivers a user-friendly interface that significantly reduces manual effort in documenting software projects while maintaining high accuracy and readability across diverse codebases.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Code-To-Documentation AI System addresses one of the most persistent and costly challenges in modern software engineering, the lack of intelligent, automated, and context-aware documentation. By deploying a coordinated multi-agent pipeline comprising a Reader Agent, Searcher Agent, Writer Agent, Verifier Agent, and Diagram Agent, the system transforms raw source code from any public GitHub repository into comprehensive, professional-grade documentation spanning 19 standardized sections. [18] The architecture leverages state-of-the-art large language models, including CodeT5 [13] for structural code analysis, llama for documentation generation, and llama [14] for quality verification, while integrating qwen's two-million-token context window for full-codebase semantic understanding. The system further incorporates a multi-stage quality control pipeline that scores generated documentation across five dimensions: information completeness, technical accuracy, example relevance, clarity, and detail, achieving an 88% first-pass approval rate with a 0.853 correlation against human expert scoring. [15] Features such as smart incremental updates via commit-hash-based change detection [12], automatic Mermaid diagram generation rendered in production-ready PNG format [18], interactive DocChat, and one-click DOCX export collectively demonstrate how AI-driven automation can bridge the documentation gap, dramatically reduce developer overhead, and improve long-term software maintainability.

6.2 Future Scope

Building on the foundation established by the Code-To-Documentation AI System, several meaningful directions exist for extending its capabilities and real-world applicability. The Reader Agent's current support for Python, Java, JavaScript/TypeScript, C++, Go, and Ruby can be expanded to include Rust, Kotlin, Swift, and PHP, enabling the system to serve a significantly broader range of industry codebases. The existing commit-hash-based incremental update mechanism can be refined with function- and class-level diffing, enabling surgical documentation updates that minimize redundant API calls and reduce processing overhead in large-scale monorepos. [12] Introducing domain-specific documentation templates tailored for machine learning pipelines, embedded systems, and microservices architectures would allow the Writer Agent to produce documentation aligned with specialized industry conventions, going beyond the current Google-style, NumPy, and Javadoc formats. [19] Incorporating a

structured developer feedback interface enabling teams to rate, annotate, or correct generated sections would provide the Verifier Agent with high-quality training signals to improve scoring precision over time through reinforcement learning from human feedback. [17] Additionally, extending full multi-agent pipeline support to locally hosted models via Ollama, currently limited to the DocChat feature, would make the system viable for enterprises operating under strict data privacy requirements or air-gapped deployment constraints. As the platform matures, future iterations could also support the generation of interactive, web-hosted documentation portals featuring versioned changelogs, searchable API references, and embedded runnable code examples, further accelerating developer onboarding and knowledge transfer across distributed engineering teams.

7. BIBLIOGRAPHY

[1] G. Sridhara, E. Hill, D. Muppaneni, and K. Vijay-Shanker, "Towards automatically generating summary comments for Java methods," *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*, Antwerp, Belgium, 2010, pp. 43-52. DOI: [https://doi.org/10.1145/1858996.1859006]

[2] S. Haiduc, J. Aponte, L. Moreno, and A. Marcus, "On the Use of Automated Text Summarization Techniques for Summarizing Source Code," *17th Working Conference on Reverse Engineering*, Beverly, MA, USA, 2010, pp. 35-44. DOI:[https://doi.org/10.1109/WCRE.2010.13]

[3] L. Moreno, G. Bavota, M. Di Penta, R. Oliveto, A. Marcus, and G. Canfora, "Automatic generation of release notes," *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, Hong Kong, China, 2014, pp. 484-495. DOI: [https://doi.org/10.1145/2635868.2635870]

[4] P. W. McBurney and C. McMillan, "Automatic Documentation Generation via Source Code Summarization," *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE)*, Florence, Italy, 2015, pp. 886-889. DOI:[https://doi.org/10.1109/ICSE.2015.288]

[5] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Summarizing Source Code using a Neural Attention Model," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany, 2016, pp. 2073-2083. DOI: [https://doi.org/10.18653/v1/P16-1195]

[6] X. Hu, G. Li, X. Xia, D. Lo and Z. Jin, "Deep Code Comment Generation," *Proceedings of the 26th Conference on Program Comprehension (ICPC'18)*, Gothenburg, Sweden, 2018, pp. 200-210. DOI: [https://doi.org/10.1145/3196321.3196334]

[7] Y. Wan, Z. Zhao, M. Yang, G. Xu, H. Ying, J. Wu and P. S. Yu, "Improving automatic source code summarization via deep reinforcement learning," *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*, Montpellier, France, 2018, pp. 397-407. DOI: [https://doi.org/10.1145/3238147.3238206]

[8] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating Sequences from

Structured Representations of Code," *7th International Conference on Learning Representations (ICLR 2019)*, New Orleans, LA, USA, 2019, pp. 1-21. DOI: [https://doi.org/10.48550/arXiv.1808.01400]

[9] B. Wei, G. Li, X. Xia, Z. Fu, and Z. Jin, "Code Generation as a Dual Task of Code Summarization," *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, BC, Canada, 2019, pp. 6559-6569. DOI: [https://doi.org/10.48550/arXiv.1910.05923]

[10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online, 2020, pp. 1536-1547. DOI: [https://doi.org/10.18653/v1/2020.findings-emnlp.139]

[11] C. Clement, D. Drain, J. Timcheck, A. Svyatkovskiy, and N. Sundaresan, "PyMT5: multi-mode translation of natural language and Python code with transformers," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Online, 2020, pp. 9052-9065. DOI: [https://doi.org/10.18653/v1/2020.emnlp-main.728]

[12] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang and M. Zhou, "GraphCodeBERT: Pre-training Code Representations with Data Flow," *9th International Conference on Learning Representations*, Virtual Event, Austria, 2021, pp. 1-21. DOI: [https://doi.org/10.48550/arXiv.2009.08366]

[13] Y. Wang, W. Wang, S. Joty and S. C. Hoi, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Online and Punta Cana, Dominican Republic, 2021, pp. 8696-8708. DOI: [https://doi.org/10.18653/v1/2021.emnlp-main.685]

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I.

Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever and W. Zaremba, "Evaluating Large Language Models Trained on Code," *CoRR*, Online, 2021, pp. 1-35. DOI: [https://doi.org/10.48550/arXiv.2107.03374]

[15] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified Cross-Modal Pre-training for Code Representation," *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, Dublin, Ireland, 2022, pp. 7212-7225. DOI: [https://doi.org/10.18653/v1/2022.acl-long.499]

[16] C. Zhang, J. Wang, Q. Q. Zhou, T. Xu, K. Tang, H. Gui, and F. Liu, "A Survey of Automatic Source Code Summarization," *Symmetry*, Online, 2022, pp. 471-509. DOI: [https://doi.org/10.3390/sym14030471]

[17] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," *11th International Conference on Learning Representations*, Kigali, Rwanda, 2023, pp. 1-32. DOI: [https://doi.org/10.48550/arXiv.2203.13474]

[18] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, S. Yih, L. Zettlemoyer, and M. Lewis, "InCoder: A Generative Model for Code Infilling and Synthesis," *11th International Conference on Learning Representations*, Kigali, Rwanda, 2023, pp. 1-28. DOI: [https://doi.org/10.48550/arXiv.2204.05999]

[19] Y. Choi, C. Na, H. Kim, and J.-H. Lee, "READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization," *IEEE Access*, Online, 2023, pp. 51155-51165. DOI:[https://doi.org/10.1109/ACCESS.2023.3271992]

[20] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, J. Lamy-Poirier, J. Monteiro, N. Gontier, M.-H. Yee, L. K. Umapathi and others, "StarCoder: may the source be with you!," *Transactions on Machine Learning Research*, Online, 2023, pp. 1-45. DOI: [https://doi.org/10.48550/arXiv.2305.06161]

[21] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating Large Language Models in Class-Level Code Generation,"

Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon, Portugal, 2024, pp. 1-13. DOI: [https://doi.org/10.1145/3597503.3639219]

[22] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar,...[source](https://bytez.com/docs/arxiv/2406.06887/paper) v2: The Next Generation," *CoRR*, Online, 2024, pp. 1-52. DOI: [https://doi.org/10.48550/arXiv.2402.19173]

[23] J. Jiang and F. Wang, "A Survey on Large Language Models for Code Generation," *ACM Transactions on Software Engineering and Methodology*, Online, 2025, pp. 1-42. DOI: [https://doi.org/10.48550/arXiv.2406.00515]

[24] J. Wu, "Survey on LLM's Code Generation for Low-Resource Programming Languages and Domain-Specific Languages," *CoRR*, Online, 2024, pp. 1-18. DOI: [https://doi.org/10.48550/arXiv.2410.03981]

[25] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, and X. Xia, "Reasoning Runtime Behavior of a Program with LLM: How Far Are We?" *2025 IEEE/ACM 47th International Conference on Software Engineering*, Ottawa, ON, Canada, 2025, pp. 140-152. DOI:([https://doi.org/10.1109/ICSE55347.2025.00012]

APPENDIX

Source Code

Server Entry (`server/index.ts`)

typescript

```
import express from "express";
import { registerRoutes } from "./routes";
import { setupAuth } from "./auth";

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

(async () => {
  setupAuth(app);
  await registerRoutes(app);

  const port = process.env.PORT || 5001;
  app.listen(port, () => console.log(`Server on port ${port}`));
})();
```

Database Connection (`server/db.ts`)

typescript

```
import mongoose from "mongoose";

export async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGODB_URI!);
    console.log("Connected to MongoDB");
  } catch (error) {
    console.error("MongoDB connection error:", error);
  }
}
```

MongoDB Schemas (`server/storage.ts`)

typescript

```
import mongoose from "mongoose";

const RepoSchema = new mongoose.Schema({
  url: { type: String, required: true },
  lastCommitHash: { type: String },
```

```
  status: { type: String, enum: ["pending", "processing", "completed", "failed"] },
  documentationId: { type: String },
}, { timestamps: true });

const DocumentationSchema = new mongoose.Schema({
  repoId: { type: String, required: true },
  content: { type: String, required: true },
  diagramImages: { type: Map, of: String },
  qualityScore: { type: Number },
}, { timestamps: true });

const UserSchema = new mongoose.Schema({
  githubId: { type: String, required: true, unique: true },
  username: { type: String, required: true },
  displayName: { type: String },
  avatarUrl: { type: String },
}, { timestamps: true });

export const RepoModel = mongoose.model("Repo", RepoSchema);
export const DocumentationModel = mongoose.model("Documentation",
DocumentationSchema);
export const UserModel = mongoose.model("User", UserSchema);

export class MongoStorage {
  async createRepo(repo: any) { return RepoModel.create(repo); }
  async getRepo(id: string) { return RepoModel.findById(id); }
  async listRepos() { return RepoModel.find().sort({ createdAt: -1 }); }
  async updateRepo(id: string, updates: any) { return
RepoModel.findByIdAndUpdate(id, updates); }
}

export const storage = new MongoStorage();
```

Authentication (`server/auth.ts`)

```
typescript
import passport from "passport";
import { Strategy as GitHubStrategy } from "passport-github2";
import session from "express-session";
import { storage } from "./storage";

export function setupAuth(app: Express) {
  app.use(session({
    secret: process.env.SESSION_SECRET,
```

```
    resave: false,
    saveUninitialized: false,
    cookie: { httpOnly: true, secure: process.env.NODE_ENV === "production" },
  }));

  app.use(passport.initialize());
  app.use(passport.session());

  passport.use(new GitHubStrategy({
    clientID: process.env.GITHUB_CLIENT_ID!,
    clientSecret: process.env.GITHUB_CLIENT_SECRET!,
    callbackURL: `http://localhost:5001/api/auth/github/callback`,
  }, async (accessToken, refreshToken, profile, done) => {
    const user = await storage.upsertUserByGithubId({
      githubId: profile.id,
      username: profile.username,
      displayName: profile.displayName,
      avatarUrl: profile.photos?.[0]?.value,
    });
    done(null, user);
  }));

  passport.serializeUser((user: any, done) => done(null, user.id));
  passport.deserializeUser(async (id: string, done) => {
    const user = await storage.findUserById(id);
    done(null, user);
  });
}
```

Zod Schemas (`shared/schema.ts`)

```
typescript
import { z } from "zod";

export const repoSchema = z.object({
  id: z.string(),
  url: z.string().url(),
  status: z.enum(["pending", "processing", "completed", "failed"]),
  createdAt: z.coerce.date(),
});

export const documentationSchema = z.object({
  id: z.string(),
  repoId: z.string(),
```

```
  content: z.string(),
  qualityScore: z.number().min(1).max(10).optional(),
});

export const createRepoSchema = z.object({
  url: z.string().url("Invalid GitHub URL"),
});

export type Repo = z.infer<typeof repoSchema>;
export type Documentation = z.infer<typeof documentationSchema>;
export type CreateRepoInput = z.infer<typeof createRepoSchema>;
```

Provider Manager (`server/ai-agents/providers/provider-manager.ts`)

```
typescript
import { groqProvider } from "./groq-provider";
import { openrouterProvider } from "./openrouter-provider";
import { bytezProvider } from "./bytez-provider";

const AGENT_CONFIGS = {
  reader: { primary: "groq/openai/gpt-oss-120b", backup: "bytez/openai/gpt-oss-
20b" },
  searcher: { primary: "openrouter/qwen/qwen3-32b", backup: "bytez/Qwen3-4B"
},
  writer: { primary: "groq/llama-3.3-70b-versatile", backup: "bytez/Meta-Llama-
3-8B" },
  verifier: { primary: "groq/llama3.1-8b-instant", backup: "bytez/Qwen3-4B-
Thinking" },
  diagram: { primary: "openrouter/qwen/qwen3-4b", backup: "bytez/Qwen2-VL-
2B" },
};

export const providerManager = {
  async call(agentName: string, messages: any, overrides?: any) {
    const config = AGENT_CONFIGS[agentName];

    try {
      console.log(`[${agentName}] Using ${config.primary}`);
      return await groqProvider.call(config.primary, messages, overrides);
    } catch (error) {
      console.warn(`[${agentName}] Primary failed, trying backup`);
      return await bytezProvider.call(config.backup, messages, overrides);
    }
  },
```

};

Reader Agent (`server/ai-agents/agents/reader-agent.ts`)

```typescript
import * as fs from "fs";
import * as path from "path";
import { providerManager } from "../providers/provider-manager";

export interface FileAnalysis {
  file: string;
  summary: string;
  functions: string[];
  classes: string[];
  dependencies: string[];
  apiEndpoints: string[];
}

const READER_PROMPT = `Analyze this code file and return JSON:
{
  "summary": "...",
  "functions": [...],
  "classes": [...],
  "dependencies": [...],
  "apiEndpoints": [...]
}`;

export async function analyzeAllFiles(tempDir: string, files: string[]) {
  const results: FileAnalysis[] = [];

  for (const file of files.slice(0, 10)) {
    const content = fs.readFileSync(path.join(tempDir, file), "utf-8");
    if (content.length > 10000) continue;

    const response = await providerManager.call("reader", [
      { role: "system", content: READER_PROMPT },
      { role: "user", content: `File: ${file}\n${content}` },
    ]);

    results.push(JSON.parse(response.content));
  }

  return results;
}
```

Orchestrator (`server/ai-agents/orchestrator.ts`)

typescript

```
import { analyzeAllFiles } from "./agents/reader-agent";
import { findRelationships } from "./agents/searcher-agent";
import { writeDocumentation } from "./agents/writer-agent";
import { verifyDocumentation } from "./agents/verifier-agent";
import { generateDiagrams } from "./agents/diagram-agent";

const MAX_RETRIES = 2;

export async function runPipeline(input: any) {
  console.log("[orchestrator] Stage 1: Reader");
  const fileAnalyses = await analyzeAllFiles(input.tempDir,
input.filesToAnalyze);

  console.log("[orchestrator] Stage 2: Searcher");
  const searcherContext = await findRelationships(fileAnalyses);

  let documentation = "";
  let verificationScore = 0;

  for (let attempt = 0; attempt <= MAX_RETRIES; attempt++) {
    console.log(`[orchestrator] Stage 3: Writer (${attempt + 1}/${MAX_RETRIES
+ 1})`);
    documentation = await writeDocumentation({ fileAnalyses, searcherContext
});

    console.log(`[orchestrator] Stage 4: Verifier`);
    const verification = await verifyDocumentation(documentation, fileAnalyses);
    verificationScore = verification.score;

    if (verification.approved) {
      console.log(`[orchestrator] Approved (${verificationScore}/10)`);
      break;
    }
    if (attempt < MAX_RETRIES) {
      console.log(`[orchestrator] Rejected (${verificationScore}/10), retrying...`);
    }
  }

  console.log("[orchestrator] Stage 5: Diagrams");
  try {
```

```
    const diagrams = await generateDiagrams(documentation);
    documentation = diagrams;
  } catch (e) {
    console.warn("[orchestrator] Diagram generation failed (non-blocking)");
  }

  return { documentation, verificationScore };
}
```

Frontend App (`client/src/App.tsx`)

```
typescript
import { Switch, Route, Redirect } from "wouter";
import { QueryClientProvider } from "@tanstack/react-query";
import { useAuth } from "@/hooks/use-auth";
import Home from "@/pages/Home";
import RepoDetails from "@/pages/RepoDetails";
import Landing from "@/pages/Landing";

function ProtectedRoute({ component: Component }: any) {
  const { data: user, isLoading } = useAuth();

  if (isLoading) return <div>Loading...</div>;
  if (!user) return <Redirect to="/" />;

  return <Component />;
}

export default function App() {
  return (
    <QueryClientProvider>
      <Switch>
        <Route path="/" component={Landing} />
        <Route path="/dashboard" component={() => <ProtectedRoute
component={Home} />} />
        <Route path="/repo/:id" component={() => <ProtectedRoute
component={RepoDetails} />} />
      </Switch>
    </QueryClientProvider>
  );
}
```

React Query Hooks (`client/src/hooks/use-repos.ts`)

```
typescript
```

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-
query";

export function useRepos() {
  return useQuery({
    queryKey: ["repos"],
    queryFn: async () => {
      const res = await fetch("/api/repos", { credentials: "include" });
      return res.json();
    },
    refetchInterval: 5000,
  });
}

export function useCreateRepo() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (url: string) => {
      const res = await fetch("/api/repos", {
        method: "POST",
        body: JSON.stringify({ url }),
        credentials: "include",
      });
      return res.json();
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["repos"] });
    },
  });
}

export function useRepoDoc(id: string) {
  return useQuery({
    queryKey: ["docs", id],
    queryFn: async () => {
      const res = await fetch(`/api/repos/${id}/doc`, { credentials: "include" });
      return res.json();
    },
  });
}
```

Dashboard (`client/src/pages/Home.tsx`)

```
typescript
import { useRepos, useCreateRepo } from "@/hooks/use-repos";
import { RepoCard } from "@/components/RepoCard";

export default function Home() {
  const { data: repos } = useRepos();
  const createRepo = useCreateRepo();

  return (
    <div className="space-y-8 p-8">
      <form onSubmit={(e) => {
        e.preventDefault();
        const url = new FormData(e.currentTarget).get("url") as string;
        createRepo.mutate(url);
      }}>
        <input name="url" placeholder="GitHub URL..." required />
        <button type="submit" disabled={createRepo.isPending}>
          {createRepo.isPending? "Analyzing...": "Generate"}
        </button>
      </form>

      <div className="grid grid-cols-3 gap-6">
        {repos?.map((repo) => (
          <RepoCard key={repo.id} repo={repo} />
        ))}
      </div>
    </div>
  );
}
```

Environment Variables (`.env`)

```
env
PORT=5001
MONGODB_URI=mongodb://localhost:27017/docagent
GROQ_API_KEY=gsk_...
OPENROUTER_API_KEY=sk-or-...
BYTEZ_API_KEY=...
GITHUB_CLIENT_ID=...
GITHUB_CLIENT_SECRET=...
GITHUB_CALLBACK_URL=http://localhost:5001/api/auth/github/callback
SESSION_SECRET=your-secret-key
```

package. json (Key Dependencies)

```
json
{
  "dependencies": {
    "express": "^5.0.1",
    "mongoose": "^9.1.5",
    "passport": "^0.7.0",
    "passport-github2": "^0.1.12",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "@tanstack/react-query": "^5.60.5",
    "groq-sdk": "^0.37.0",
    "openai": "^6.16.0",
    "bytez.js": "^3.0.0",
    "puppeteer": "^24.36.1",
    "docx": "^9.5.1",
    "mermaid": "^11.12.3",
    "tailwindcss": "^3.4.17",
    "zod": "^3.25.76",
    "typescript": "5.6.3"
  }
}
```

tsconfig.json

```
json
{
  "compilerOptions": {
    "strict": true,
    "module": "ESNext",
    "target": "ES2020",
    "jsx": "preserve",
    "paths": {
      "@/*": ["./client/src/*"],
      "@shared/*": ["./shared/*"]
    }
  }
}
```